

QILIN

v2

Meicheng Liu¹, Xiaobin Yu¹, Zhongyi Zhang², Jiangyu Chen¹, Shichang Wang³, Dongdai Lin¹

June 30, 2026

¹State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, People's Republic of China

²School of Cryptologic Science and Engineering, Shandong University, People's Republic of China

³Nanyang Technological University, Singapore

Contents

1	Specification	3
1.1	Notations	3
1.2	Construction	3
1.3	Permutation	3
1.3.1	Linear Diffusion Layer	6
1.3.2	The Nonlinear Layer	6
1.3.3	The Row and Column Rotations	6
1.3.4	Addition of Constants	7
2	Features	8
3	Design Rationale	9
3.1	Construction	9
3.2	Design of the Nonlinear Layer	9
3.2.1	Composition of Two S-boxes	9
3.2.2	The Complementing Transform	12
3.3	Design of the Linear Diffusion Layer	12
3.3.1	Higher Branch Number	12
3.3.2	Fewer Rotate Operations	12
3.3.3	Lower Bound of Differential Trail Weight	15
3.3.4	Choice of the Parameter of Linear Layer	17
3.4	Choice of the Round Numbers	17
4	Security Claims and Analysis	18
4.1	Security Claims	18
4.2	Analysis of the Construction	18
4.2.1	The Sponge Construction	18
4.2.2	Length Extension Attack	18
4.2.3	Free-Start Attack	18
4.3	Analysis of the Permutation	18
4.3.1	Differential and Linear Properties	18
4.3.2	Algebraic Properties	22
4.3.3	S-box Linearization	23
4.3.4	Expected Size of Collision Subsets for the $\mathcal{S}$ S-box	24
4.3.5	Integral Distinguishers and Division Property	26
4.3.6	Zero-Sum Distinguisher	26
4.3.7	Statistical Randomness Testing	27
4.4	Analysis of the Algorithm	27
4.4.1	Collision Attack	27
4.4.2	Preimage attack	29
4.4.3	Quantum Security	30

5	Implementation	33
5.1	Software Implementation	33
5.1.1	X86.64 Architecture	33
5.1.2	ARM Architecture	33
5.2	Hardware Implementation	34

Chapter 1

Specification

QILIN is a family of cryptographic functions, especially including hash functions. This chapter defines the QILIN hash functions.

1.1 Notations

Table 1.1: Notation

$\lceil n \rceil$	Ceiling function, the smallest integer $\geq n$
0^k	Bitstring of k bits (variable length if $k = *$), all 0
$ x $	Length of the bitstring x in bits
$[x]_k$	Bitstring x truncated to the first (most significant) k bits
$x y$	Concatenation of bitstrings x and y
$x \oplus y$	Bitwise XOR of x and y
$x \odot y$	Bitwise AND of x and y
$x \wedge y$	Bitwise AND of x and y
$x \vee y$	Bitwise OR of x and y
$x_{i,j}$	The 64-bit word in position (i, j) , $0 \leq i, j < 7$
S_r	The first r bits of S
S_c	The last c bits of S

1.2 Construction

The structure of QILIN adopts the sponge construction with an initial value IV , as illustrated in Figure 1.1 and specified in Algorithm 1, where r denotes the bitrate and c denotes the capacity of the sponge construction. When it is used as a cryptographic hash function, the initial value IV is required to be a fixed value. For the QILIN hash functions, we set $IV = 0^{c/2}$.

1.3 Permutation

The permutation QILIN- f operates on a state of size $7 \times 7 \times 64 = 3136$ bits, as shown in Figure 1.2a. The state $S = x_{0,0}||x_{1,0}||x_{2,0}||\dots||x_{6,6}$ can be seen as a 7×7 array, where $x_{i,j}$'s are 64-bit words for $0 \leq i, j \leq 6$. The number of rounds, denoted by N_r , is set to 12, 14, and 16 for the permutation of QILIN-512, QILIN-768, and QILIN-1024, respectively. QILIN- f consists of N_r rounds of the transformation p . Each transformation p consists of five steps, including a linear diffusion layer $\mathcal{L}$, row rotation $\rho_{\mathcal{Q}}$, nonlinear layer $\mathcal{Q}$, column rotation $\rho_{\mathcal{L}}$ and addition of constants $\mathcal{C}$, that is,

$$p = \mathcal{C} \circ \rho_{\mathcal{L}} \circ \mathcal{Q} \circ \rho_{\mathcal{Q}} \circ \mathcal{L}.$$

QILIN- f is defined as shown in Algorithm 2. The first four steps of p are shown in Figure 1.2.

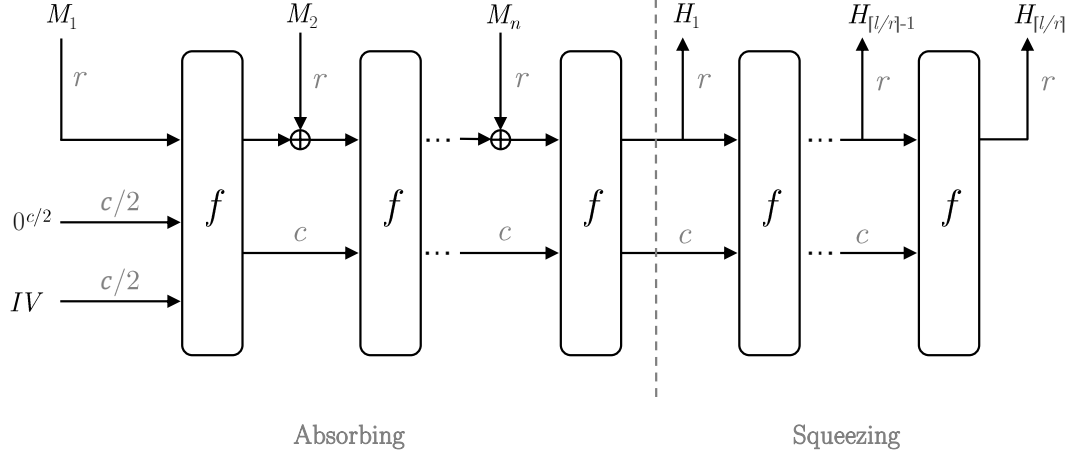


Figure 1.1: The sponge construction with IV

Algorithm 1 The sponge construction with IV

Input: message $M \in \{0, 1\}^*$, output bit length l

Output: hash $H \in \{0, 1\}^l$

Absorbing

$\bar{M}_1, \dots, \bar{M}_n \leftarrow M \| 1 \| 0^* \| 1$

$S \leftarrow f(\bar{M}_1 \| 0^{c/2} \| IV)$

for $i = 2, \dots, n$ **do**

$S \leftarrow f((S_r \oplus \bar{M}_i) \| S_c)$

Squeezing

$H_1 \leftarrow S_r$

for $i = 2, \dots, t = \lceil l/r \rceil$ **do**

$S \leftarrow f(S)$

$H_i \leftarrow S_r$

return $\lfloor H_1 \| \dots \| H_t \rfloor_l$

Algorithm 2 The permutation QILIN- f

for $t = 0, \dots, N_r - 1$ **do**

$state \leftarrow R_t(state)$

The round function R_t :

for $j = 0, \dots, 6$ **do**

$(y_{0,j}, y_{1,j}, \dots, y_{6,j}) \leftarrow L(x_{0,j}, x_{1,j}, \dots, x_{6,j})$

$\triangleright \mathcal{L}$

for $i = 0, \dots, 6$ **do**

$(y_{i,0}, y_{i,1}, \dots, y_{i,6}) \leftarrow (y_{i,-i}, y_{i,1-i}, \dots, y_{i,6-i})$

$\triangleright \rho_Q$

for $i = 0, \dots, 6$ **do**

$(x_{i,0}, x_{i,1}, \dots, x_{i,6}) \leftarrow Q(y_{i,0}, y_{i,1}, \dots, y_{i,6})$

$\triangleright Q$

for $j = 0, \dots, 6$ **do**

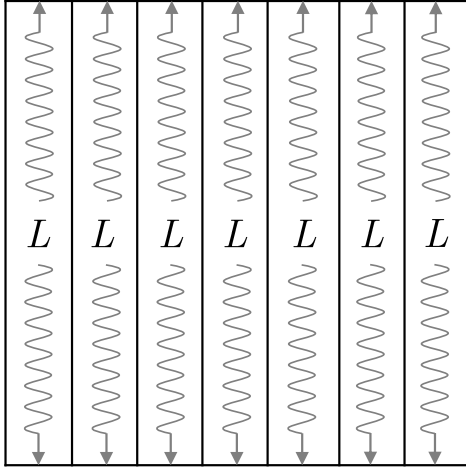
$(x_{0,j}, x_{1,j}, \dots, x_{6,j}) \leftarrow (x_{-j,j}, x_{1-j,j}, \dots, x_{6-j,j})$

$\triangleright \rho_{\mathcal{L}}$

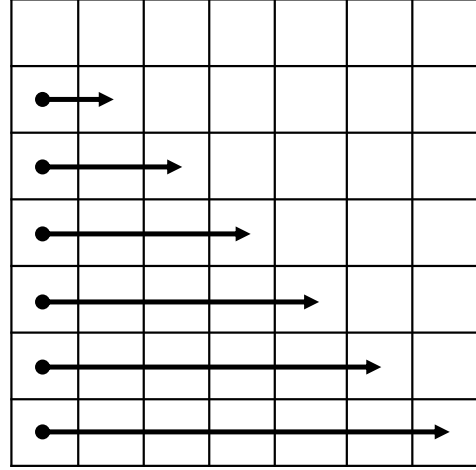
$x_{0,0} \leftarrow x_{0,0} \oplus C_t$

$x_{0,0}$	$x_{0,1}$	$x_{0,2}$	$x_{0,3}$	$x_{0,4}$	$x_{0,5}$	$x_{0,6}$
$x_{1,0}$	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$	$x_{1,6}$
$x_{2,0}$	$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$	$x_{2,5}$	$x_{2,6}$
$x_{3,0}$	$x_{3,1}$	$x_{3,2}$	$x_{3,3}$	$x_{3,4}$	$x_{3,5}$	$x_{3,6}$
$x_{4,0}$	$x_{4,1}$	$x_{4,2}$	$x_{4,3}$	$x_{4,4}$	$x_{4,5}$	$x_{4,6}$
$x_{5,0}$	$x_{5,1}$	$x_{5,2}$	$x_{5,3}$	$x_{5,4}$	$x_{5,5}$	$x_{5,6}$
$x_{6,0}$	$x_{6,1}$	$x_{6,2}$	$x_{6,3}$	$x_{6,4}$	$x_{6,5}$	$x_{6,6}$

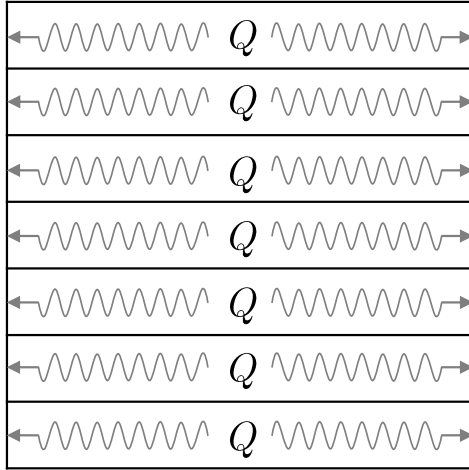
(a) The 3136-bit state, $7 \times 7 \times 64$



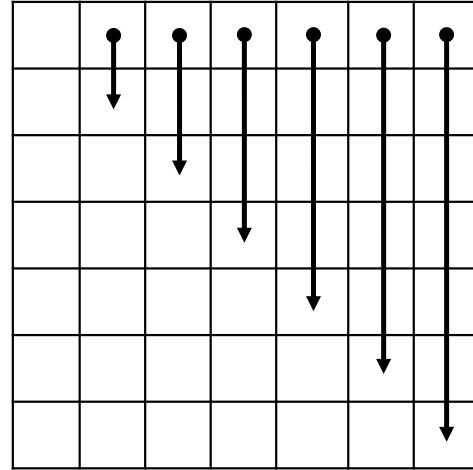
(b) The Linear Diffusion Layer $\mathcal{L}$



(c) The Row Rotation ρ_Q



(d) The Nonlinear Layer $\mathcal{Q}$



(e) The Column Rotation $\rho_{\mathcal{L}}$

Figure 1.2: The 3136-bit state, linear layer, row rotation, nonlinear layer and column rotation.

1.3.1 Linear Diffusion Layer

The linear diffusion layer $\mathcal{L}$ is composed of 7 parallel linear layers L . The operation $x \oplus (y \lll z)$ is called shifted variant of XOR or shifted XOR. We use 28 shifted XORs to construct the linear layer L , as shown in Algorithm 3. The parameters n_1, n_2 and m is shown in Table 1.2.

Algorithm 3 The linear layer L

Input: $(y_0, y_1, \dots, y_6)$
for $i = 7, \dots, 34$ **do**
 $y_i \leftarrow y_{n_1[i]} \oplus (y_{n_2[i]} \lll m_i)$
return $(y_{28}, y_{29}, \dots, y_{34})$

Table 1.2: The parameters of linear layer L

i	7	8	9	10	11	12	13	14	15	16	17	18	19	20
$n_1[i]$	5	2	4	1	6	3	0	7	10	11	14	8	16	12
$n_2[i]$	2	4	1	6	3	0	7	5	12	9	13	15	17	18
m_i	0	0	0	0	0	0	1	39	23	18	46	54	5	15

i	21	22	23	24	25	26	27	28	29	30	31	32	33	34
$n_1[i]$	9	13	11	8	18	19	20	24	22	25	23	21	26	27
$n_2[i]$	19	20	21	22	23	24	24	22	25	23	21	26	27	20
m_i	26	33	46	44	9	30	0	0	0	0	0	0	1	39

1.3.2 The Nonlinear Layer

The nonlinear layer $\mathcal{Q}$ updates the state via $7 \times 64 = 448$ parallel applications of the 7-bit S-box. It is implemented using 7 parallel bitsliced Q mappings, with all operations performed on 64-bit words, as detailed in Algorithm 4. The 7-bit Sbox is shown in Figure 1.3.

Algorithm 4 The nonlinear mapping Q (bitsliced implementation)

Input: $(a_0, a_1, \dots, a_6)$
 $t_0 = (a_1 \wedge a_2) \oplus a_0$
 $t_1 = (a_2 \vee a_3) \oplus a_1$
 $t_2 = (a_3 \wedge a_4) \oplus a_2$
 $t_3 = (a_4 \vee a_5) \oplus a_3$
 $t_4 = (a_5 \wedge a_6) \oplus a_4$
 $t_5 = (\overline{a_6} \wedge a_0) \oplus a_5$
 $t_6 = (a_0 \vee a_1) \oplus a_6$
 $b_0 = (t_0 \wedge t_5) \oplus t_2$
 $b_1 = (t_5 \vee t_3) \oplus t_0$
 $b_2 = (t_3 \wedge t_1) \oplus t_5$
 $b_3 = (t_1 \vee t_6) \oplus t_3$
 $b_4 = (t_6 \wedge t_4) \oplus t_1$
 $b_5 = (t_4 \wedge t_2) \oplus t_6$
 $b_6 = (t_2 \vee t_0) \oplus t_4$
return $(b_0, b_1, \dots, b_6)$

1.3.3 The Row and Column Rotations

The row rotation $\rho_{\mathcal{Q}}$ and column rotation $\rho_{\mathcal{L}}$ consists of the same mapping $(1, \rho, \rho^2, \dots, \rho^6)$ with rotation operation $\rho(x_0, x_1, \dots, x_6) = (x_6, x_0, \dots, x_5)$. The row rotation $\rho_{\mathcal{Q}}$ is performed on the rows, i.e., the inputs to the nonlinear layer $\mathcal{Q}$, while the column rotation $\rho_{\mathcal{L}}$ is performed on the columns, i.e., the inputs to the linear layer $\mathcal{L}$ of the following round (if exists).

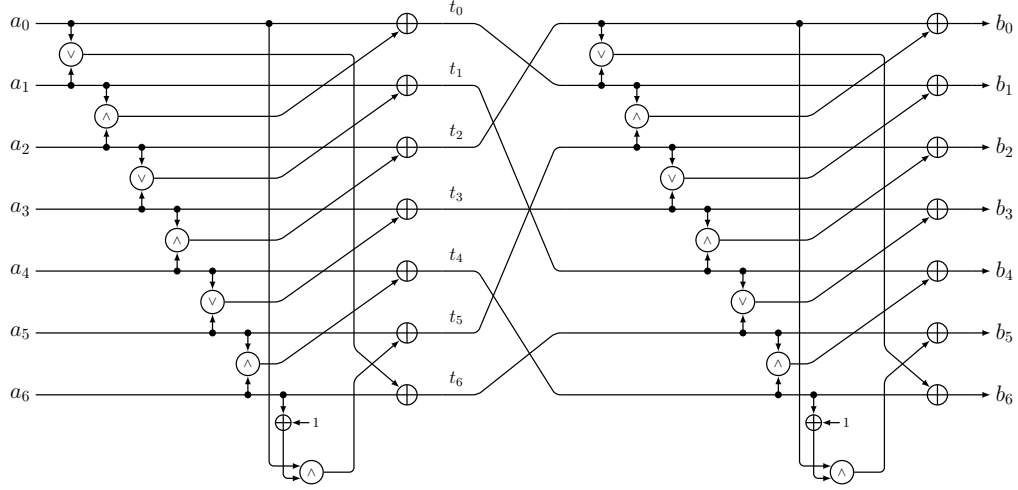


Figure 1.3: The Sbox

1.3.4 Addition of Constants

The constant addition step adds a round constant C_t to 64-bit word $x_{0,0}$ in round t :

$$x_{0,0} \leftarrow x_{0,0} \oplus C_t.$$

The round constants C_t are shown in Table 1.3 and generated using a 15-bit Linear Feedback Shift Register (LFSR). The LFSR is defined over $GF(2)$ by the primitive polynomial:

$$P(x) = x^{15} + x^{14} + x^2 + x + 1.$$

Let the state of the LFSR at round t be $S_{LFSR}^{(t)} = (s_t, s_{t-1}, s_{t-2}, \dots, s_{t-14})$. The LFSR is initialized with $S_{LFSR}^{(0)} = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1)$ and updated by $S_{LFSR}^{(t+1)} = (s_{t+1}, s_t, s_{t-1}, \dots, s_{t-13})$ with:

$$s_{t+1} = s_t \oplus s_{t-1} \oplus s_{t-13} \oplus s_{t-14}.$$

The i -th bit of C_t is denoted by $C_t[i]$. The 15 bits of the LFSR state are mapped to the 15 bits of C_t :

$$\begin{aligned} C_t[0] &= s_t, & C_t[1] &= s_{t-1}, & C_t[2] &= s_{t-2}, & C_t[4] &= s_{t-3}, & C_t[6] &= s_{t-4}, \\ C_t[7] &= s_{t-5}, & C_t[8] &= s_{t-6}, & C_t[9] &= s_{t-7}, & C_t[10] &= s_{t-8}, & C_t[11] &= s_{t-9}, \\ C_t[12] &= s_{t-10}, & C_t[13] &= s_{t-11}, & C_t[14] &= s_{t-12}, & C_t[17] &= s_{t-13}, & C_t[24] &= s_{t-14}. \end{aligned}$$

Table 1.3: Round constants C_t

t	Constant C_t	t	Constant C_t
0	0000000001027fd7	8	0000000001027d95
1	0000000001027fd6	9	0000000001027b53
2	0000000001027fd5	10	00000000010276c6
3	0000000001027fd3	11	0000000001026d95
4	0000000001027fc6	12	0000000001025b53
5	0000000001027f95	13	00000000010236c6
6	0000000001027f53	14	0000000001006d95
7	0000000001027ec6	15	0000000000025b52

Chapter 2

Features

- **Structural Simplicity via the Sponge Construction:** Adopting the sponge construction ensures a clean and elegant design. This framework provides well-understood indifferentiability properties, simplifying both the security proof and the final implementation.
- **Flexibility of Components:** Our design methodology provides high flexibility in constructing permutations with diverse state sizes, i.e., $7 \times 7 \times 64$, $7 \times 7 \times 32$ or $7 \times 7 \times 16$. Specifically, the linear layer can be generalized to accommodate various sizes of bits, i.e., 7×64 , 7×32 or 7×16 .
- **Composite S-box Architecture:** We employ a composite S-box construction and have conducted a comprehensive study on the differential uniformity (DU) of such cascaded structures. This ensures that the non-linear layer provides robust resistance against differential cryptanalysis while remaining computationally efficient.
- **Reuse of Core Component for Small Hardware Area:** To minimize the hardware area, our design promotes structural reuse. The S-box is constructed by cascading two same χ_n -like S-boxes. The linear layer $L = p_1 \circ p_2 \circ p_1$ can reuse p_1 . By sharing hardware resources for repeated components, the total area is significantly reduced.
- **Multi-platform Architectural Advantages:** Our design exhibits significant performance benefits across diverse computational platforms. On ARM-based architectures, the rotation and XOR operations can be fused into a single instruction cycle by leveraging the barrel shifter, thereby substantially accelerating the execution of the linear layer. For x86.64 architectures, we have optimized the number of rotate operations by setting several offsets to zero. Furthermore, by employing the lane complementing transform techniques, we have successfully reduced the number of NOT operations required in the S-box implementation. In terms of hardware synthesis (ASIC/FPGA), the algorithm relies exclusively on primitive logic gates such as XOR, AND, and OR, ensuring high efficiency.
- **Parallelism-Friendly Design:** The proposed linear diffusion layer is composed of seven identical linear transformation units. This structural symmetry is highly conducive to SIMD (Single Instruction, Multiple Data) parallelization. Specifically, utilizing the AVX-512 instruction set, seven 64-bit words can be processed concurrently, leading to a high-throughput implementation of the linear layer.
- **Provability of Differential Security:** The proposed permutation is easy to facilitate the proof of lower bound on differential trail weight. By simplifying the modeling of weight propagation, we provide a straightforward yet rigorous proof of high lower bounds for differential weights, ensuring a verifiable security margin against differential cryptanalysis. This provides a higher level of confidence in the security margins against differential cryptanalysis.
- **Functional Versatility:** Beyond its primary role as a cryptographic hash function, the sponge construction with IV serves as a versatile cryptographic primitive. Due to the inherent properties of the sponge construction, the design can be extended to support various symmetric-key functionalities, including Authenticated Encryption with Associated Data (AEAD), Message Authentication Codes (MAC), and Pseudo-Random Number Generators (PRNG).

Chapter 3

Design Rationale

3.1 Construction

We adopted the sponge construction for QILIN because of flexibility, simplicity and provable security. We use an IV of $c/2$ bits based on the proof in [NO14]. We will explain this in Section 4.2.3.

3.2 Design of the Nonlinear Layer

3.2.1 Composition of Two S-boxes

The 7-bit S-box employed in our design can be regarded as a composition of two 7-bit S-boxes. To construct a 7-bit S-box with low differential uniformity using a minimal number of operations and a simpler structure, we investigate the differential uniformity of S-boxes formed by the composition of two S-boxes. We begin by introducing several formal definitions.

Notation 1. For any $\mathcal{S} : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$, and any differential (a, b) , with $a, b \in \mathbb{F}_2^n$, we denote the input (resp. output) set of (a, b) as follows:

$$\text{XDDT}_{\mathcal{S}}(a, b) := \{x \in \mathbb{F}_2^n | \mathcal{S}(a + x) + \mathcal{S}(x) = b\} \text{ and } \text{YDDT}_{\mathcal{S}}(a, b) := \{\mathcal{S}(x) | x \in \text{XDDT}_{\mathcal{S}}(a, b)\}.$$

Moreover, $\text{DDT}_{\mathcal{S}}(a, b) := \#\text{XDDT}_{\mathcal{S}}(a, b) = \#\text{YDDT}_{\mathcal{S}}(a, b)$.

Definition 1 (Planar differential [DR07]). A differential (a, b) for $\mathcal{S} : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^s$ is said to be planar if both sets $\text{XDDT}_{\mathcal{S}}(a, b)$ and $\text{YDDT}_{\mathcal{S}}(a, b)$ are affine subspaces. In this case, we use the following notation:

$$\text{XDDT}_{\mathcal{S}}(a, b) := x_0^{a,b} + \text{U}_{\mathcal{S}}(a, b) \text{ and } \text{YDDT}_{\mathcal{S}}(a, b) := y_0^{a,b} + \text{V}_{\mathcal{S}}(a, b) \quad (3.1)$$

where $y_0^{a,b} = \mathcal{S}(x_0^{a,b})$.

Definition 2 (Input-planar differential). A differential (a, b) for $\mathcal{S} : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^s$ is said to be input-planar if set $\text{XDDT}_{\mathcal{S}}(a, b)$ is affine subspace. In this case, we use the following notation:

$$\text{XDDT}_{\mathcal{S}}(a, b) := x_0^{a,b} + \text{U}_{\mathcal{S}}(a, b) \quad (3.2)$$

Definition 3 (Output-planar differential). A differential (a, b) for $\mathcal{S} : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^s$ is said to be output-planar if set $\text{YDDT}_{\mathcal{S}}(a, b)$ is affine subspace. In this case, we use the following notation:

$$\text{YDDT}_{\mathcal{S}}(a, b) := y_0^{a,b} + \text{V}_{\mathcal{S}}(a, b) \quad (3.3)$$

Theorem 1. Let us consider an S-box $\mathcal{S}$ obtained as the composition of two S-boxes, S_1 and S_2 , i.e., $\mathcal{S} = S_2 \circ S_1$. If (a_1, a_2) is an output-planar differential for S_1 and (a_2, a_3) is an input-planar differential for S_2 , we have

$$\text{DP}_{\mathcal{S}}(a_1, a_2, a_3) = \begin{cases} \text{DP}_{S_1}(a_1, a_2) \text{DP}_{S_2}(a_2, a_3) 2^{\dim(E)} & \text{if } (x_0^{a_2, a_3} + y_0^{a_1, a_2}) \in E^\perp \\ 0 & \text{otherwise,} \end{cases} \quad (3.4)$$

where $E = (\text{V}_{S_1}(a_1, a_2) + \text{U}_{S_2}(a_2, a_3))^\perp$.

Proof. First we compute

$$\begin{aligned}
\text{DP}_S(a_1, a_2, a_3) &= \sum_{u_2} (D_{(u_2, a_2)(0, a_1)}^{S_1} D_{(0, a_3)(u_2, a_2)}^{S_2}) \\
&= \sum_{u_2} ((-1)^{u_2 y_0^{a_1, a_2}} \text{DDT}_{S_1}(a_1, a_2) 2^{-n} \mathbf{1}_{u_2 \in (\mathbf{V}_{S_1}(a_1, a_2))^\perp} \times \\
&\quad (-1)^{u_2 x_0^{a_2, a_3}} \text{DDT}_{S_2}(a_2, a_3) 2^{-n} \mathbf{1}_{u_2 \in (\mathbf{U}_{S_2}(a_2, a_3))^\perp}) \\
&= \frac{\text{DDT}_{S_1}(a_1, a_2)}{2^n} \frac{\text{DDT}_{S_2}(a_2, a_3)}{2^n} \sum_{u_2 \in E} (-1)^{u_2(x_0^{a_2, a_3} + y_0^{a_1, a_2})}.
\end{aligned}$$

For any vector space E , we have

$$\sum_{x \in E} (-1)^{c \cdot x} = \begin{cases} 2^{\dim(E)} & \text{if } c \in E^\perp \\ 0 & \text{otherwise.} \end{cases} \quad (3.5)$$

Then,

$$\text{DP}(a_1, a_2, a_3) = \begin{cases} \frac{\text{DDT}_{S_1}(a_1, a_2)}{2^n} \frac{\text{DDT}_{S_2}(a_2, a_3)}{2^n} 2^{\dim(E)} & \text{if } (x_0^{a_2, a_3} + y_0^{a_1, a_2}) \in E^\perp \\ 0 & \text{otherwise,} \end{cases} \quad (3.6)$$

□

If E is the zero space, the nonzero probability attains its minimum value of $\frac{\text{DDT}_{S_1}(a_1, a_2)}{2^n} \frac{\text{DDT}_{S_2}(a_2, a_3)}{2^n}$. In this case, the sum of $\mathbf{V}_{S_1}(a_1, a_2)$ and $\mathbf{U}_{S_2}(a_2, a_3)$ spans the entire space.

Lemma 1. The n -bit χ operation denoted by $\chi_n : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ is defined as following:

$$y_i = x_i + \overline{x_{i+1}} x_{i+2} \quad \text{for } i \in [0, n-1]$$

where $X = (x_0, x_1, \dots, x_{n-1})$ and $Y = (y_0, y_1, \dots, y_{n-1})$ denote the n -bit input and output, respectively. We have that any differential (a, b) of χ_n are input-planar.

Proof. For an input-output pair (X, Y) satisfying the differential (a, b) , we define that $X \lll m = (x_m, x_{m+1}, \dots, x_{m-1})$. We have

$$\begin{aligned}
Y &= X \oplus \overline{(X \lll 1)} \odot (X \lll 2), \\
Y \oplus b &= (X \oplus a) \oplus \overline{((X \oplus a) \lll 1)} \odot ((X \oplus a) \lll 2).
\end{aligned}$$

Taking the XOR of these two equations yields

$$b = a \oplus ((a \lll 1) \odot (X \lll 2)) \oplus ((X \lll 1) \odot (a \lll 2)) \oplus ((a \lll 1) \odot (a \lll 2)) \oplus (a \lll 2).$$

Similarly, for another input-output pair (X', Y') satisfying the same differential, we have

$$b = a \oplus ((a \lll 1) \odot (X' \lll 2)) \oplus ((X' \lll 1) \odot (a \lll 2)) \oplus ((a \lll 1) \odot (a \lll 2)) \oplus (a \lll 2).$$

XORing these two expressions gives

$$0 = ((a \lll 1) \odot ((X \oplus X') \lll 2)) \oplus (((X \oplus X') \lll 1) \odot (a \lll 2)).$$

$\text{XDDT}_{\chi_n}(a, b)$ satisfies the properties required for an affine space. Therefore, (a, b) is input-planar. □

Moreover, by the proof of Lemma 1, it follows that $\text{DDT}_{\chi_n}(a, b) = \text{DDT}_{\chi_n}(a, b')$. And we have

$$(a \lll 1) \odot ((X \oplus X') \lll 2) = (X \oplus X') \lll 1 \odot (a \lll 2).$$

Assume that $a = (a_0, a_1, \dots, a_{n-1})$ and $X \oplus X' = (t_0, t_1, \dots, t_{n-1})$, we have

$$\begin{aligned} a_1 t_0 &= a_0 t_1, \\ a_2 t_1 &= a_1 t_2, \\ &\dots \\ a_0 t_{n-1} &= a_{n-1} t_0. \end{aligned}$$

Therefore, for a differential (a, b) , we have

$$\text{DDT}_{\chi_n}(a, b) \begin{cases} = 2^n & w(a) = 0 \\ = 2^{n-2} & w(a) = 1 \\ \leq 2^{n-3} & w(a) > 1 \end{cases} \quad (3.7)$$

where $w(a)$ is the hamming weight of a . When $w(a) = 1$, there are four differentials (a, b) , (a, b') , (a, b'') , (a, b''') and one of $w(b)$, $w(b')$, $w(b'')$, $w(b''')$ is equal to 1.

Definition 4 (Permutation-XOR Equivalence Class). *The permutation-XOR equivalence class of S-box $\mathcal{S} : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ is defined as following:*

$$\mathcal{E}(\mathcal{S}) = \{\mathcal{S}' : \mathcal{S}'(x) = \pi_{out}(\mathcal{S}(\pi_{in}(x) \oplus a)) \oplus b, a, b \in \mathbb{F}_2^n, \pi_{out}, \pi_{in} \in P_n\}.$$

where P_n denotes the group of $n \times n$ permutation matrices.

The π operation is free in implementation. We study the the composition of two S-boxes which are permutation-XOR equivalent to χ_n . Since applying permutations and XOR with constant vectors to the input or output does not affect the differential uniformity (DU), it suffices to study S-boxes of the form

$$\mathcal{S} = \chi_n(\pi(\chi_n) \oplus c).$$

Definition 5 (Quasidifferential transition matrix [BR22]). *Let $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$. Its quasidifferential transition matrix is the $(2^{2n} \times 2^{2n})$ -matrix defined by*

$$D^F(v, b)(u, a) := 2^{-n} \sum_{x \in \text{XDDT}_F(a, b)} (-1)^{v \cdot F(x) + u \cdot x}$$

where $x \cdot y$ denotes the scalar product between x and y .

Let us consider an S-box $\mathcal{S}$ obtained as the composition of two S-boxes, S_1 and S_2 , i.e., $\mathcal{S} = S_2 \circ S_1$. $S_1 = \pi(\chi_n \oplus c)$, $S_2 = \chi_n$. (a_1, a_2) is a differential for S_1 and (a_2, a_3) is a differential for S_2 . We have proved that (a_2, a_3) is input-planar. By [BR22, Theorem 3.2(3)], [BR22, Theorem 4.1] and [CF25, Lemma 2], we can prove that the differential probability of (a_1, a_2, a_3)

$$\begin{aligned} Pr(a_1, a_2, a_3) &= \sum_{u_2} (D_{(u_2, a_2)(0, a_1)}^{S_1} D_{(0, a_3)(u_2, a_2)}^{S_2}) \\ &= 2^{-n} \frac{\text{DDT}_{S_2}(a_2, a_3)}{2^n} \sum_{u_2} \sum_{y \in \text{YDDT}_{S_1}(a_1, a_2)} (-1)^{u_2 \cdot y} (-1)^{u_2 \cdot x_0^{a_2, a_3}} \mathbf{1}_{u_2 \in (U_{S_2}(a_2, a_3))^\perp} \\ &= 2^{-n} \frac{\text{DDT}_{S_2}(a_2, a_3)}{2^n} \sum_{y \in \text{YDDT}_{S_1}(a_1, a_2)} \sum_{u_2 \in (U_{S_2}(a_2, a_3))^\perp} (-1)^{u_2 \cdot (y + x_0^{a_2, a_3})} \end{aligned}$$

where

$$\sum_{u_2 \in (U_{S_2}(a_2, a_3))^\perp} (-1)^{u_2 \cdot (y + x_0^{a_2, a_3})} = \begin{cases} \frac{2^n}{\text{DDT}_{S_2}(a_2, a_3)} & \text{if } (y + x_0^{a_2, a_3}) \in U_{S_2}(a_2, a_3) \\ 0 & \text{otherwise.} \end{cases} \quad (3.8)$$

Because DU of χ_n is 2^{n-2} , we investigate the differential with the highest differential probability. If $\text{DDT}_{S_2}(a_2, a_3) = 2^{n-2}$, then there are other three differentials that satisfy $\text{DDT}_{S_2}(a_2, a'_3) = \text{DDT}_{S_2}(a_2, a''_3) = \text{DDT}_{S_2}(a_2, a'''_3) = 2^{n-2}$. We want to minimize the maximum among these four DDT.

Ideally, the y in $\text{YDDT}(a_1, a_2)$ are uniformly distributed among the four affine spaces: $\text{XDDT}_{S_2}(a_2, a_3)$, $\text{XDDT}_{S_2}(a_2, a'_3)$, $\text{XDDT}_{S_2}(a_2, a''_3)$, $\text{XDDT}_{S_2}(a_2, a'''_3)$. Now,

$$\Pr(a_1, a_2, a_3) = \Pr(a_1, a_2, a'_3) = \Pr(a_1, a_2, a''_3) = \Pr(a_1, a_2, a'''_3) = \frac{\text{DDT}_{S_1}(a_1, a_2)}{2^{n+2}}.$$

When $\text{DDT}_{S_1}(a_1, a_2) = 2^{n-2}$, $\Pr(a_1, a_2, a_3) = 2^{-4}$. We have

$$\text{DDT}_{S_2 \circ S_1}(a_1, a_3) = 2^n \sum_{a_2} \Pr(a_1, a_2, a_3).$$

For any $w(a_1) = 1$, there exists a differential (a_1, a_2) such that $w(a_2) = 1$ and $\text{DDT}_{S_1}(a_1, a_2) = 2^{n-2}$. There is at least one a_2 such that $\text{DDT}_{S_1}(a_1, a_2) = \text{DDT}_{S_2}(a_2, a_3) = 2^{n-2}$. Ideally, there is only one a_2 and $\text{DDT}_{S_2 \circ S_1}(a_1, a_3) = 2^{n-4}$.

Consequently, under this configuration, the optimal differential uniformity (DU) for S-boxes of form $\chi_n(\pi(\chi_n) \oplus c)$ is 2^{n-4} . By exhaustively searching through 7-bit S-boxes of this form, we verified that this optimal DU is achievable.

3.2.2 The Complementing Transform

[BDP⁺12] introduces an implementation technique called the lane complementing transform, which, by using the law of De Morgan ($\bar{a} \wedge \bar{b} = \overline{a \vee b}$), reduces the number of NOT operations required when implementing χ_n . However, the permutation's input and output need to be corrected by applying masks. In our design, we employ this technique by directly using the χ_n with reduced NOT operations instead of the original χ_n , thereby we don't need to apply masks.

3.3 Design of the Linear Diffusion Layer

In our previous work [YL25], we introduced a family of linear layers, termed Multiple Rows Mixers (MRM), which feature high differential branch numbers and low XOR counts. Now, we present a refined construction of MRM that achieves further improvements.

3.3.1 Higher Branch Number

In [YL25], a linear layer with a high differential branch number was constructed by composing two layers, p_1 and p_2 . This design achieved an XOR count per bit of 2.6, with a differential branch number of 12 and a linear branch number of 5.

We observe that p_1 can also be leveraged to enhance the linear branch number. Based on this insight, we propose a new structure: $p_1 \circ p_2 \circ p_1$. This modification significantly improves the linear branch number with only a small increase in computational cost. The resulting linear layer achieves an XOR count per bit of 4, while the differential branch number is increased to 22 and linear branch number is increased to 17.

Furthermore, this structure is highly efficient for hardware implementation. The p_1 layer can be reused to reduce the circuit area.

3.3.2 Fewer Rotate Operations

We refine the structure of p_1 as shown in Algorithm 5 and Table 3.1. we denote $\mathcal{N}_L[d]$ as the minimum number of active input bits for the linear layer L when the number of active output bits is d . Our goal is to maintain a high $\mathcal{N}_L[1]$ value while keeping as many shift parameters m_i as possible at zero.

Algorithm 5 The linear layer p_1

Input: $(x_0, x_1, \dots, x_{r-1})$
for $i = r, \dots, 2r$ **do**
 $x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i)$
return $(x_{r+1}, x_{r+2}, \dots, x_{2r})$

Table 3.1: The parameters of p_1

i	$r+1$	$r+2$	$\dots$	r	$\dots$	$b+r$	$\dots$	$2r-1$	$2r$
$n_1[i]$	0	1	$\dots$	a	$\dots$	b	$\dots$	$r-1$	r
$n_2[i]$	1	2	$\dots$	$a+1$	$\dots$	$b+1$	$\dots$	r	0
m_i	m_1	m_2	$\dots$	m_{a+1}	$\dots$	m_{b+1}	$\dots$	m_r	m_0

Let's consider the case that the number of active output bits is one. Assume that $x_{b+r} = x_b \oplus (x_{b+1} \lll m_{b+1})$ contains only one active bit and other outputs are zero. That is the following expression has only one active bit:

$$(x_a \lll (\sum_{i=0}^a m_i - m_{a+1})) \oplus (x_a \lll (-m_{a+1})) \oplus (x_a \lll (\sum_{i=b+2}^r m_i + \sum_{i=0}^a m_i)) \quad (3.9)$$

We can set $a = 0$. Then the Expression 3.9 is equivalent to

$$(x_0 \lll (m_0 - m_1)) \oplus (x_0 \lll (-m_1)) \oplus (x_0 \lll (\sum_{i=b+2}^r m_i + m_0)) \quad (3.10)$$

where $1 \leq b \leq r-1$. To minimize the number of non-zero parameters, we can set m_i to 0 for $1 \leq i \leq r-1$. The expression is equivalent to

$$x_0 \oplus (x_0 \lll m_r) \oplus (x_0 \lll (64 - m_0)) \quad (3.11)$$

under cyclic shift. In this case, only two out of the $r+1$ shift parameters are non-zero (m_0 and m_r).

Next, we determine the optimal values for m_0 and m_r .

This optimization problem is equivalent to selecting the rotation offsets a and b for the bitwise function $y = x \oplus (x \lll a) \oplus (x \lll b)$, which shares a structural similarity with the Σ functions in the SHA-2 family and the linear layer of the Ascon permutation.

Specifically, the cryptographic metric $\mathcal{N}_l[1]$ for the linear mapping $l: y = x \oplus (x \lll a) \oplus (x \lll b)$ corresponds to the Hamming weight of the multiplicative inverse of the trinomial $f(x) = 1 + x^a + x^b$ within the quotient ring $R_n = \mathbb{F}_2[x]/\langle x^{2^n} + 1 \rangle$, where $0 < a < b < 2^n$.

Since the coefficient field has characteristic 2, the algebraic structure of R_n is heavily governed by the Frobenius endomorphism. Under the linear property of the power map in characteristic 2, the 2^n -th power of any polynomial in R_n simplifies. Applying this property to the trinomial $f(x)$ yields:

$$f(x)^{2^n} = (1 + x^a + x^b)^{2^n} = 1 + x^{a \cdot 2^n} + x^{b \cdot 2^n}. \quad (3.12)$$

Because the relation $x^{2^n} \equiv 1$ holds in R_n , it follows that $x^{k \cdot 2^n} \equiv 1 \pmod{x^{2^n} + 1}$ for any non-negative integer k . Consequently, the expression simplifies to $f(x)^{2^n} \equiv 1 + 1 + 1 \equiv 1 \pmod{x^{2^n} + 1}$, which implies that $f(x)$ is always invertible in R_n . The multiplicative inverse is uniquely given by $g(x) = f(x)^{2^n-1}$.

To analyze the structural properties of $g(x)$, we decompose the exponent $2^n - 1$ using its binary expansion, $2^n - 1 = \sum_{i=0}^{n-1} 2^i$. This standard factorization allows us to express $g(x)$ as a product of n sparse polynomials:

$$g(x) \equiv \prod_{i=0}^{n-1} (1 + x^{a \cdot 2^i} + x^{b \cdot 2^i}) \pmod{x^{2^n} + 1}. \quad (3.13)$$

Let $\text{wt}(g(x))$ denote the Hamming weight of $g(x)$, defined as the number of non-zero coefficients in its canonical representative of degree less than 2^n . Expanding the product formulation reveals that the exponent of each term in the unreduced expansion is of the form $\sum_{i=0}^{n-1} c_i 2^i$, where each coefficient c_i is restricted to the digit set $\{0, a, b\}$. Since the coefficient arithmetic is performed over $\mathbb{F}_2$, the coefficient of x^k (for $0 \leq k < 2^n$) in the reduced polynomial corresponds to the parity of the number of ways to represent k . Let $N(k, t)$ denote the number of such representations defined by:

$$N(k, t) = \# \left\{ (c_0, c_1, \dots, c_{t-1}) \in \{0, a, b\}^t : \sum_{i=0}^{t-1} c_i 2^i \equiv k \pmod{2^n} \right\}. \quad (3.14)$$

The Hamming weight $\text{wt}(g(x))$ is therefore equivalent to the number of integers $k \in \{0, 1, \dots, 2^n - 1\}$ for which $N(k, n)$ is odd. This formulation effectively reformulates the algebraic problem of polynomial inversion into a combinatorial analysis of restricted digit representations modulo 2^n .

Case 1: Both a and b are even. In this scenario, the representation count $N(k, n)$ can be isomorphically reduced to a lower-dimensional case where at least one of the exponents is odd.

Case 2: One exponent is odd and the other is even. Without loss of generality, we assume a is odd and b is even. By exploiting the cyclic shift invariance of the operation, the relation $y = x \oplus (x \lll a) \oplus (x \lll b)$ can be shifted right cyclically by a bits, which is equivalent to multiplying the polynomial relation by $x^{-a} \equiv x^{2^n - a} \pmod{x^{2^n} + 1}$. This transforms the original relation into:

$$y \ggg a = x \oplus (x \lll (b - a)) \oplus (x \lll (2^n - a)). \quad (3.15)$$

Since a is odd and b is even, both resulting shift offsets, $b - a$ and $2^n - a$, are odd. Consequently, the mixed-parity case is successfully reduced to the case where both exponents are odd.

Case 3: Both a and b are odd. Based on the aforementioned reductions, it suffices to restrict our subsequent analysis to the case where both a and b are odd. Without loss of generality, we assume $0 < a < b$. Let $v_2(x)$ denote the 2-adic valuation of x , defined as the exponent of the highest power of 2 dividing x . Let $v = v_2(b - a)$ and write $b = a + 2^v \alpha$, where α is an odd integer. For $i \geq n - v$, we have $2^i a \equiv 2^i b \pmod{2^n}$. This congruence yields:

$$N(k, n) \equiv N(k, n - v) \pmod{2}. \quad (3.16)$$

Case 3.1: $v \geq n/2$. For any two distinct tuples $(c_0, c_1, \dots, c_{n-v-1})$ and $(c'_0, c'_1, \dots, c'_{n-v-1}) \in \{0, a, b\}^{n-v}$, the difference between their corresponding linear combinations satisfies:

$$\sum_{i=0}^{n-v-1} (c_i - c'_i) 2^i \equiv A \cdot a + B \cdot 2^v \alpha \pmod{2^n}, \quad (3.17)$$

where $|A|, |B| \leq 2^{n-v} - 1$. Since $v_2(A \cdot a) \leq n - v - 1 \leq v - 1$ and $v_2(B \cdot 2^v \alpha) \in \{0\} \cup [v, n - 1]$, and given that A and B cannot both be zero, it follows that $A \cdot a + B \cdot 2^v \alpha \not\equiv 0 \pmod{2^n}$. Thus, we have:

$$\sum_{i=0}^{n-v-1} c_i 2^i \not\equiv \sum_{i=0}^{n-v-1} c'_i 2^i \pmod{2^n}. \quad (3.18)$$

Consequently, $N(k, n - v) = 1$ for any k that can be expressed in the form $k \equiv \sum_{i=0}^{n-v-1} c_i 2^i \pmod{2^n}$, and $N(k, n - v) = 0$ otherwise. This unique representation implies that $\text{wt}(g(x)) = 3^{n-v}$.

Case 3.2: $v < n/2$.

Case 3.2.1: $a + b = 2^n$. For the symmetric case where the exponents satisfy $a + b = 2^n$, assuming without loss of generality that a is odd, the mapping $\sigma(x) = x^a$ defines a ring automorphism over R_n . Since this automorphism preserves the multiplicative structure and acts as a permutation on the exponent set, the Hamming weight of the multiplicative inverse $g(x) = (1 + x^a + x^{2^n - a})^{-1}$ is invariant and equivalent to that of the base case $a = 1$:

$$\text{wt} \left((1 + x^a + x^{2^n - a})^{-1} \right) = \text{wt} \left((1 + x + x^{2^n - 1})^{-1} \right). \quad (3.19)$$

We now derive the exact weight formula by analyzing the base case $f_1(x) = 1 + x + x^{-1}$, whose inverse is given by:

$$g_1(x) \equiv \prod_{i=0}^{n-2} (1 + x^{2^i} + x^{-2^i}) \pmod{x^{2^n} + 1}. \quad (3.20)$$

Let S_k denote the set of exponents (with non-zero coefficients in $\mathbb{F}_2$) of the partial product $A_k(x) = \prod_{i=0}^{k-1} (1 + x^{2^i} + x^{-2^i})$ for $1 \leq k \leq n - 1$. The set S_k evolves recursively as:

$$S_k = S_{k-1} \cup (S_{k-1} + 2^{k-1}) \cup (S_{k-1} - 2^{k-1}). \quad (3.21)$$

Due to coefficient arithmetic over $\mathbb{F}_2$, terms with overlapping exponents cancel out. An exponent e cancels if and only if there exist $x, y \in S_{k-1}$ such that $x - y = 2^{k-1}$. By induction on k , the number of such overlapping pairs is exactly determined, yielding the following recurrence relation for the cardinality $W_k = |S_k|$:

$$W_k = 2W_{k-1} + (-1)^k, \quad (3.22)$$

with the initial condition $W_1 = |\{0, 1, -1\}| = 3$. Solving this linear recurrence relation yields the closed-form solution:

$$W_k = \frac{2^{k+2} - (-1)^k}{3}. \quad (3.23)$$

Since the full inverse $g(x)$ corresponds to the partial product of $n-1$ terms, the final Hamming weight w_n is:

$$w_n = W_{n-1} = \frac{2^{n+1} - (-1)^{n-1}}{3}, \quad (3.24)$$

which is precisely the $(n+1)$ -th Jacobsthal number J_{n+1} .

Case 3.2.2: $a + b \neq 2^n$. Although an explicit closed-form expression for $\text{wt}(g(x))$ is not directly available in these scenarios, we derive a recurrence relation that enables a more computationally efficient evaluation of $N(k, t)$:

$$N(k, t) = \begin{cases} 0, & \text{if } k \neq 0, t = 0, \\ 1, & \text{if } k = 0, \\ N\left(\frac{k}{2}, t-1\right) + N\left(\frac{k+2^n}{2}, t-1\right), & \text{if } k \text{ is even,} \\ N(k-a, t) + N(k-b, t), & \text{if } k \text{ is odd.} \end{cases} \quad (3.25)$$

In conclusion, we summarize our results in Table 3.2. Furthermore, we investigate the value of $\mathcal{N}_l[2]$ in these two specific cases.

Table 3.2: The values of $\mathcal{N}_l[1]$ and $\mathcal{N}_l[2]$ for the linear mapping $l: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ defined by $y = x \oplus (x \lll a) \oplus (x \lll b)$ with $0 < a < b$ and odd a, b . Let $v = v_2(b-a)$, the highest power of 2 dividing $b-a$.

		$v < n/2$	
		$a + b = 2^n$	$a + b \neq 2^n$
$\mathcal{N}_l[1]$	3^{n-v}	$\frac{2^{n+1} - (-1)^{n-1}}{3}$	No closed-form expression
$\mathcal{N}_l[2]$	$2 \times 3^{n-v-1}$	2	No closed-form expression

Notably, high values of $\mathcal{N}_l[1]$ and $\mathcal{N}_l[2]$ can be achieved when $v \geq n/2$. Based on this analysis, we set m_r to be an odd number, set $64 - m_0 - m_r$ to be an odd multiples of 8 ($2^{n/2} = 8$), and set all other m_i values to be 0. The construction of p_1 is shown in figure 3.1.

3.3.3 Lower Bound of Differential Trail Weight

Lemma 2. *The permutation QILIN-f consists of linear diffusion layers and substitution layer. The substitution layer is applied vertically across the 448 64-bit words using 7-bit S-boxes, while the linear diffusion layer $\mathcal{L}$ is composed of 7 horizontal linear layers L .*

Suppose L satisfies the following bit-level differential property: for any non-zero difference Δ_{in} input to L with the number of active bits $w_b(\Delta_{in}) \leq x$, the number of active bits in the output difference $\Delta_{out} = L(\Delta_{in})$ satisfies $w_b(\Delta_{out}) \geq y$.

Then, $\mathcal{L}$ inherits a corresponding S-box-level property: for any non-zero differential characteristic, if the number of active S-boxes related to the input of $\mathcal{L}$ is at most x , then the number of active S-boxes related to the output of $\mathcal{L}$ is at least y .

Proof. Let the input state of $\mathcal{L}$ be denoted by $S = (s_0, s_1, \dots, s_6)$, where each $s_i \in \mathbb{F}_2^{448}$ represents the input to the i -th instance of L . We define $w_s(S)$ as the number of active S-boxes of S .

Assume that $0 < w_s(S) \leq x$. By the definition of the S-box weight, this implies that for each active s_i , its internal bit-level Hamming weight satisfies $w_b(s_i) \leq w_s(S) \leq x$.

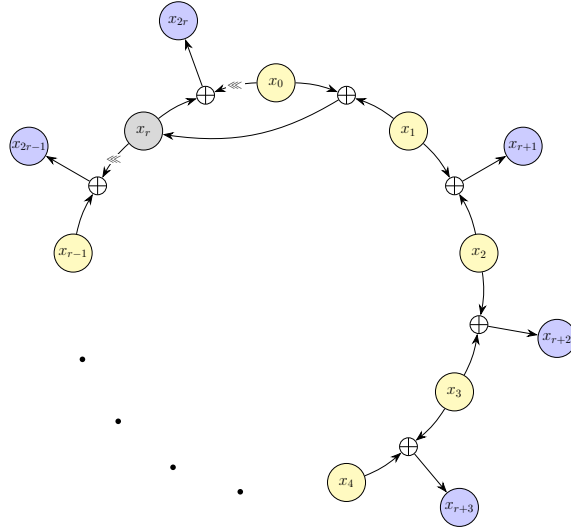


Figure 3.1: The linear layer p_1 . The yellow nodes represent the inputs, the blue represent the outputs, and the gray node represents the temporary variable.

Since $S \neq 0$, there exists at least one index $j \in \{0, \dots, 6\}$ such that $s_j \neq 0$. For this specific component, the bit-level bound of L applies:

$$w_b(s_j) \leq x \implies w_b(L(s_j)) \geq y.$$

Let the output state of $\mathcal{L}$ be $S' = (L(s_0), L(s_1), \dots, L(s_6))$. We have:

$$w_s(S') \geq w_b(L(s_j)) \geq y.$$

Thus, the S-box-level bound $w_s(S') \geq y$ is established. $\square$

Lemma 3. Suppose the linear layer L satisfies:

$$w_b(\Delta_{out}) \geq \begin{cases} \tau & \text{if } 1 \leq w_b(\Delta_{in}) \leq \tau \\ 2\tau - w_b(\Delta_{in}) & \text{if } w_b(\Delta_{in}) \geq \tau + 1 \end{cases} \quad (3.26)$$

For any non-zero r -round differential trail of QILIN- f , the total number of active S-boxes b_r satisfies:

$$b_r \geq \tau(r - 1) + 1 \quad (3.27)$$

Proof. Let a_i be the number of active S-boxes in round i . By the property of L and Lemma 2, we have

$$a_i \geq \begin{cases} \tau & \text{if } 1 \leq a_{i-1} \leq \tau \\ \tau - 1 & \text{if } a_{i-1} \leq \tau + 1 \\ \tau - 2 & \text{if } a_{i-1} \leq \tau + 2 \\ \dots & \\ 1 & \text{if } a_{i-1} \leq 2\tau - 1 \end{cases} \quad (3.28)$$

Therefore, we have

$$a_i \geq \begin{cases} \tau & \text{if } 1 \leq a_{i-1} \leq \tau \\ 2\tau - a_{i-1} & \text{if } a_{i-1} \geq \tau + 1 \end{cases} \quad (3.29)$$

Let $b_i = \sum_{1 \leq k \leq i} a_k$ denote the cumulative weight up to round i . For $i \geq 3$, we consider two cases:

1. If $a_{i-1} \geq \tau$, then $a_{i-1} + a_i \geq a_{i-1} + (2\tau - a_{i-1}) = 2\tau$, leading to $b_i \geq b_{i-2} + 2\tau$.
2. If $a_{i-1} < \tau$, then $a_i \geq \tau$, leading to $b_i \geq b_{i-1} + \tau$.

Thus, the recurrence relation is $b_i \geq \min\{b_{i-2} + 2\tau, b_{i-1} + \tau\}$. By induction on r , we have:

$$b_r \geq \min\{b_1 + \tau(r-1), b_2 + \tau(r-2)\} \quad (3.30)$$

Given that $b_1 = a_1 \geq 1$, and for $r = 2$, $b_2 = a_1 + a_2 \geq \tau + 1$ (since $a_1 \in [1, \tau] \implies a_2 \geq \tau$), we obtain:

$$b_r \geq \min\{1 + \tau(r-1), \tau + 1 + \tau(r-2)\} = \tau(r-1) + 1 \quad (3.31)$$

□

The S-box employed in our design is characterized by a differential uniformity of $DU = 8$ and a minimum differential weight of $w_{\min} = 4$. Consequently, establishing that the linear layer L satisfies the weight propagation constraints defined in Eq. (3.26) is sufficient to prove that the lower bound for the r -round differential trail weight, denoted as W_r , is:

$$W_r \geq w_{\min} \times (\tau(r-1) + 1) = 4\tau(r-1) + 4. \quad (3.32)$$

The verification of the properties of L can be efficiently performed by modeling the linear layer using Mixed Integer Linear Programming (MILP).

This evaluation strategy offers three primary advantages:

- **Simplified constraints:** By converting the constraints on active S-boxes into bit-level activity constraints, the MILP modeling process is significantly simplified. Modeling the number of active bits of linear layers is computationally more efficient than modeling the number of active S-boxes.
- **Reduction in the state size:** Our approach only requires modeling a 7×64 linear layer, rather than the full 49×64 state. This reduction in the number of variables and constraints leads to a substantial decrease in the time required for the MILP solver to converge.
- **Modeling without S-box:** This strategy bypasses the intricate modeling of S-box differential properties. Due to the superior diffusion characteristics of our designed linear layer, a high differential trail bound can be guaranteed even without imposing explicit constraints on the S-box.

3.3.4 Choice of the Parameter of Linear Layer

The method proposed in [YL25] provides a theoretical framework for designing permutations with large states. However, as the state size increases, the parameter space—especially for the shift values m —expands significantly, making the original random search strategy computationally expensive. To overcome this limitation, we use an iterative search strategy based on guided local correction.

The process begins by using SAT or MILP solvers to identify differential trails with low weights. We trace these trails through the linear layer to locate the first instance of bitwise cancellation (i.e., where $1 \oplus 1 = 0$). We then increment the corresponding shift parameter $m[i]$ by 1 and restart the search. If the cancellation persists at the same position in the next iteration, we skip it and target the next cancellation point.

The differential and linear mask search procedures are executed alternately. This iterative refinement continues until the branch numbers for both differential and linear properties meet the design requirements.

3.4 Choice of the Round Numbers

Based on our evaluation, the hash functions built on 6 rounds of the permutation offer sufficient protection against collision attacks and preimage attacks. We adopt 12 or more rounds for high security margins.

Chapter 4

Security Claims and Analysis

4.1 Security Claims

Table 4.1: Security claims for recommended parameter configurations of QILIN

Algorithm	State size	Output size	r	c	Security		
					Coll.	2nd pre.	Pre.
QILIN-512	3136	512	2112	1024	256	512	512
QILIN-768	3136	768	1600	1536	384	768	768
QILIN-1024	3136	1024	1088	2048	512	1024	1024

4.2 Analysis of the Construction

4.2.1 The Sponge Construction

The structure of QILIN is based on the sponge construction. Our construction achieves the same indistinguishability from a Random Oracle as the sponge construction [BDPA08]. It is also resistant to standard cryptographic attacks, including collision attacks, preimage attacks and second preimage attacks.

4.2.2 Length Extension Attack

By taking the advantage of the sponge construction, the QILIN hash functions are resistant to the length extension attacks.

4.2.3 Free-Start Attack

The construction of QILIN has an IV of length $c/2$. It is proved in [NO14] that PHOTON (a sponge-like algorithm) with keeping the same indistinguishable security, where the length of the first message block is modified from r bits to $r + c/2$ bits. Therefore, QILIN is secure even if the $c/2$ -bit IV is under the control of the attacker.

4.3 Analysis of the Permutation

4.3.1 Differential and Linear Properties

S-box

The differential uniformity is 8 and the linearity is 40. The DDT and LAT are shown in Figure 4.1 and Figure 4.2.

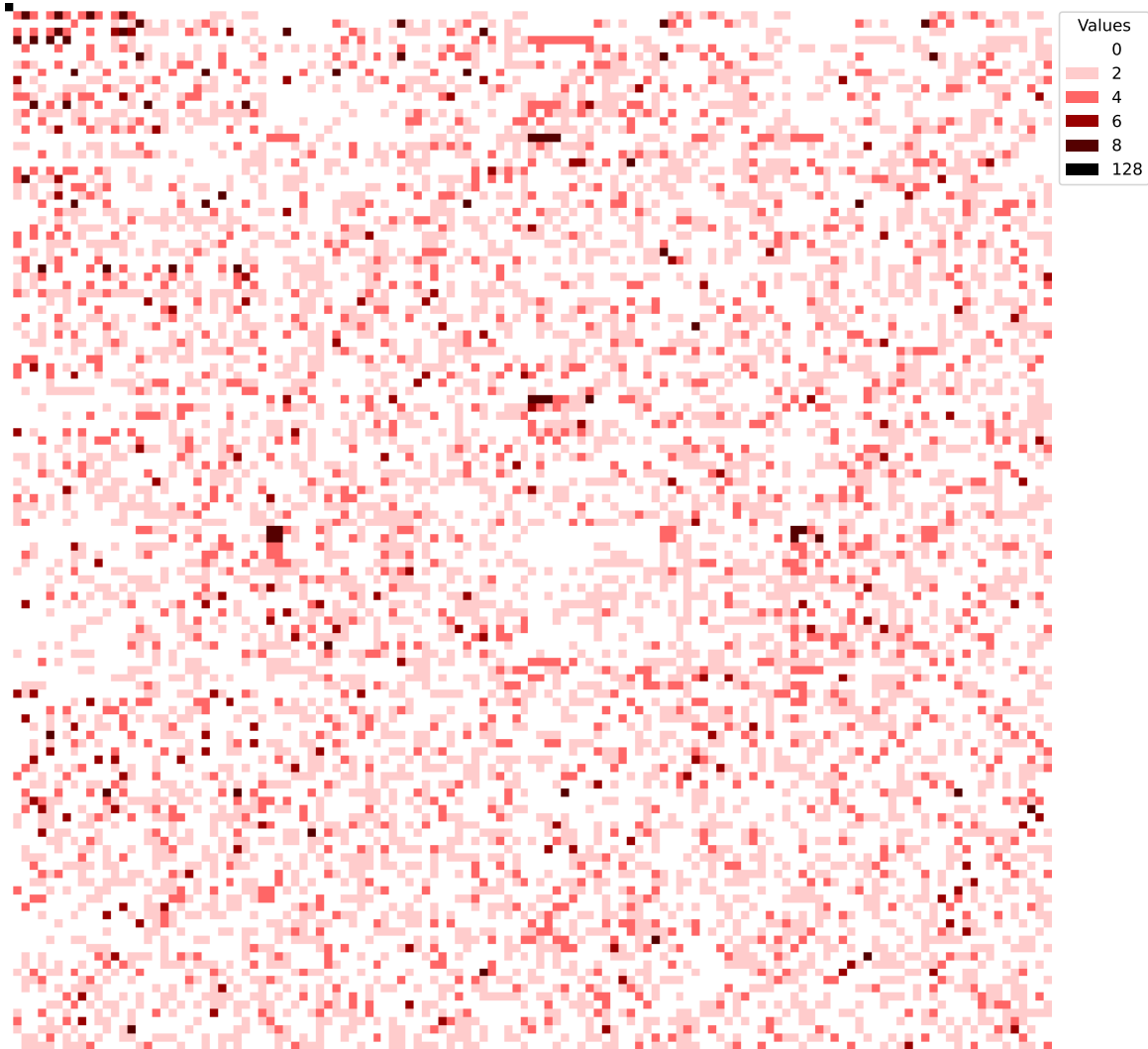


Figure 4.1: Differential distribution table

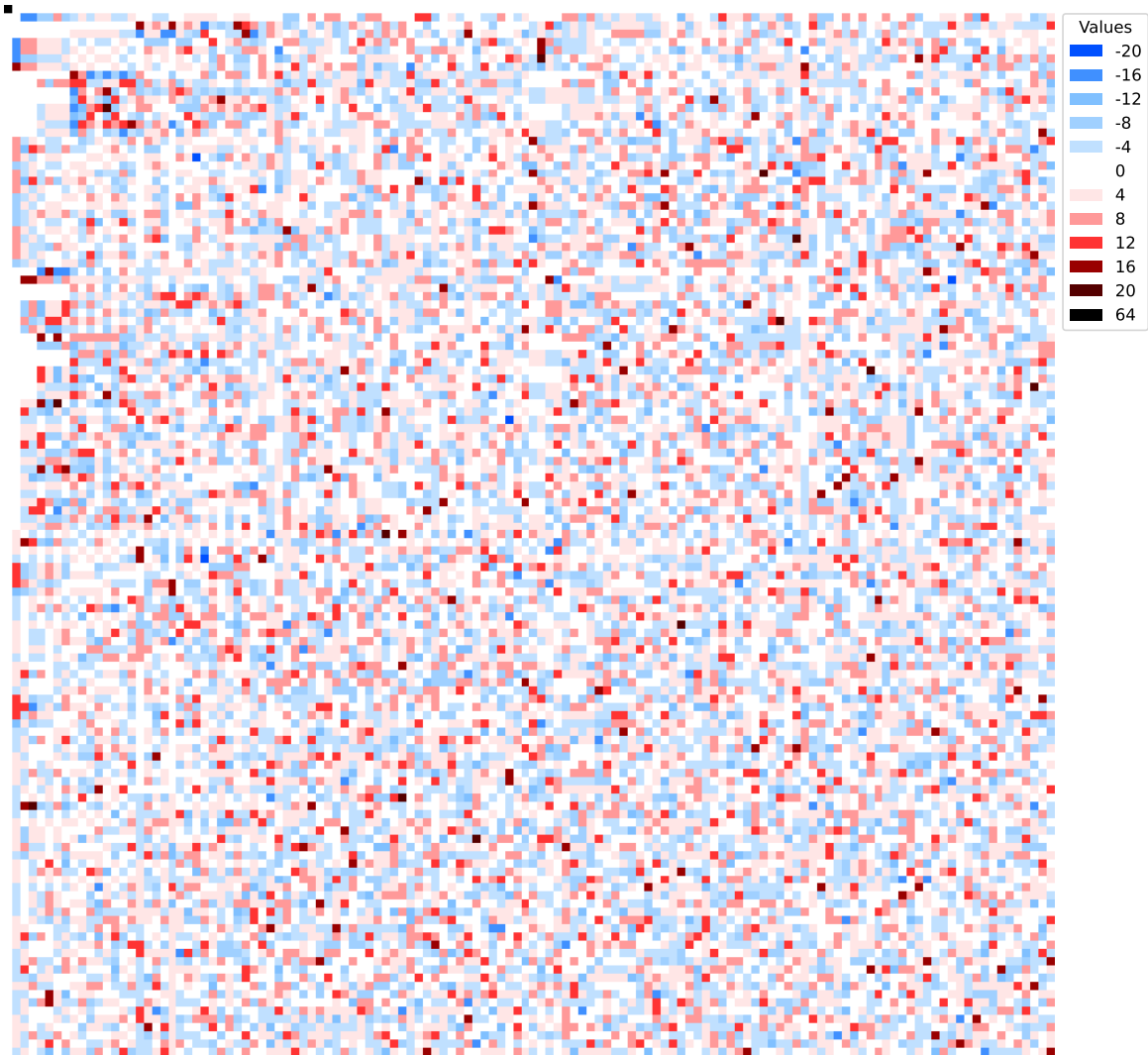


Figure 4.2: Linear approximation table

Linear layer

The differential branch number is 22 and the linear branch number is 17.

Bound of trail weight

Based on the methodology presented in Section 3.3.3, we introduce more stringent constraints into our model for computing the lower bound on the weight of a differential trial.

Table 4.2: Constraints of active S-boxes over two rounds of QILIN- f

$\#AS_i$	$\#AS_{i+1}$
1	≥ 24
≤ 5	≥ 20
≤ 7	≥ 16
≤ 13	≥ 14
≤ 15	≥ 13
≤ 17	≥ 11
≤ 18	≥ 9
≤ 20	≥ 8
≤ 24	≥ 7
≤ 42	≥ 6
≤ 83	≥ 5
≥ 84	≥ 1

We employ MILP to model the number of active S-boxes in each round and derive the lower bounds for the differential trail weight as shown in Table 4.3.

Table 4.3: The lower bound of minimum differential trail weight

Round	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Lower bound	88	120	176	228	284	336	392	444	500	552	608	660	716	768	824

Activity pattern

We investigate the activation patterns of linear layers, focusing on the propagation properties of active bits. Let $w(x)$ denote the Hamming weight of the input x , representing the number of active bits. Furthermore, $w(L(x))$ and $w(L^{-1}(x))$ represent the Hamming weights of the outputs after the transformation by the linear layer L and its inverse L^{-1} , respectively. These values are shown in Table 4.4. For the transformation L^T by the transpose of L and its inverse $(L^T)^{-1}$, the values of the Hamming weights are shown in Table 4.5.

Table 4.4: Activity pattern of the linear layer L

$w(x)$	$w_{min}(L(x))$	$w_{min}(L^{-1}(x))$
1	24	205
2	20	130
3	20	152
4	20	84
5	20	102
6	16	25
7	18	[19,43]
8	14	[18,40]
9	14	[18,33]
10	16	[18,24]
11	16	16
12	14	16
13	15	14
14	13	8
15	13	12
16	11	6

Table 4.5: Activity pattern of the transpose of the linear layer L

$w(x)$	$w_{min}(L^T(x))$	$w_{min}((L^T)^{-1}(x))$
1	24	216
2	25	153
3	14	156
4	16	70
5	13	34
6	11	11

4.3.2 Algebraic Properties

Algebraic Normal Form (ANF)

The substitution layer $\mathcal{Q}$ is composed of 7×64 parallel S-boxes. Let $x_0, x_1, \dots, x_6$ denote the input bits and $y_0, y_1, \dots, y_6$ denote the output bits. The ANF of the S-box is given by:

$$\begin{aligned}
y_0 &= x_0x_1x_2x_6 + x_1x_2x_5 + x_0x_1x_2 + x_3x_4 + x_0x_6 + x_0x_5 + x_2 + x_0, \\
y_1 &= x_0x_4x_5x_6 + x_0x_5x_6 + x_0x_4x_6 + x_0x_4x_5 + x_0x_3x_6 + x_4x_5 + x_3x_5 + x_1x_2 + x_0x_6 + x_0x_5 + x_0x_4 \\
&\quad + x_0x_3 + x_5 + x_4 + x_3, \\
y_2 &= x_2x_3x_4x_5 + x_3x_4x_5 + x_2x_4x_5 + x_2x_3x_5 + x_2x_3x_4 + x_1x_4x_5 + x_3x_5 + x_3x_4 + x_2x_5 + x_2x_4 + x_1x_5 \\
&\quad + x_1x_4 + x_1x_3 + x_0x_6 + x_5 + x_3 + x_0, \\
y_3 &= x_0x_1x_2x_3 + x_2x_3x_6 + x_1x_2x_3 + x_0x_2x_3 + x_0x_1x_3 + x_0x_1x_2 + x_4x_5 + x_3x_6 + x_2x_6 + x_2x_3 + x_1x_6 \\
&\quad + x_1x_3 + x_1x_2 + x_0x_3 + x_0x_2 + x_0x_1 + x_6 + x_5 + x_4 + x_2 + x_1 + x_0, \\
y_4 &= x_0x_1x_5x_6 + x_1x_5x_6 + x_0x_5x_6 + x_0x_1x_4 + x_5x_6 + x_4x_6 + x_2x_3 + x_1x_4 + x_0x_4 + x_3 + x_2 + x_1, \\
y_5 &= x_3x_4x_5x_6 + x_2x_5x_6 + x_2x_4 + x_0x_1 + x_6 + x_2 + x_1 + x_0, \\
y_6 &= x_1x_2x_3x_4 + x_0x_3x_4 + x_5x_6 + x_3x_4 + x_0x_2 + x_4 + x_2 + x_0.
\end{aligned}$$

The algebraic degree of the S-box is 4, with each linear combination of y_i reaching this maximum degree.

The ANF of the inverse of the S-box is given by:

$$\begin{aligned}
x_0 = & y_1 y_2 y_3 y_4 y_6 + y_0 y_2 y_3 y_4 y_6 + y_2 y_3 y_4 y_6 + y_2 y_3 y_4 y_5 + y_1 y_2 y_5 y_6 + y_1 y_2 y_4 y_6 + y_1 y_2 y_3 y_6 + y_0 y_2 y_4 y_5 \\
& + y_0 y_2 y_3 y_6 + y_0 y_2 y_3 y_5 + y_0 y_2 y_3 y_4 + y_0 y_1 y_4 y_5 + y_0 y_1 y_3 y_5 + y_0 y_1 y_3 y_4 + y_4 y_5 y_6 + y_3 y_5 y_6 \\
& + y_3 y_4 y_6 + y_2 y_5 y_6 + y_2 y_4 y_6 + y_2 y_4 y_5 + y_2 y_3 y_6 + y_1 y_4 y_5 + y_1 y_3 y_5 + y_1 y_3 y_4 + y_1 y_2 y_6 + y_1 y_2 y_5 \\
& + y_1 y_2 y_4 + y_0 y_5 y_6 + y_0 y_4 y_6 + y_0 y_4 y_5 + y_0 y_3 y_5 + y_0 y_3 y_4 + y_0 y_2 y_5 + y_0 y_2 y_4 + y_0 y_2 y_3 + y_0 y_1 y_5 \\
& + y_0 y_1 y_4 + y_0 y_1 y_3 + y_5 y_6 + y_4 y_6 + y_4 y_5 + y_3 y_6 + y_2 y_6 + y_2 y_3 + y_1 y_5 + y_1 y_4 + y_1 y_3 + y_1 y_2 \\
& + y_0 y_6 + y_0 y_3 + y_0 y_2 + y_0 y_1 + y_6 + y_5 + y_4 + y_3, \\
x_1 = & y_1 y_3 y_4 y_5 y_6 + y_0 y_2 y_3 y_5 y_6 + y_3 y_4 y_5 y_6 + y_2 y_3 y_5 y_6 + y_1 y_4 y_5 y_6 + y_1 y_3 y_5 y_6 + y_1 y_2 y_5 y_6 + y_1 y_2 y_4 y_5 \\
& + y_0 y_3 y_5 y_6 + y_0 y_3 y_4 y_6 + y_0 y_2 y_4 y_5 + y_0 y_2 y_3 y_5 + y_0 y_1 y_5 y_6 + y_0 y_1 y_3 y_4 + y_4 y_5 y_6 + y_3 y_4 y_6 \\
& + y_2 y_5 y_6 + y_2 y_3 y_5 + y_1 y_3 y_5 + y_1 y_2 y_5 + y_1 y_2 y_4 + y_1 y_2 y_3 + y_0 y_5 y_6 + y_0 y_4 y_6 + y_0 y_3 y_5 + y_0 y_2 y_6 \\
& + y_0 y_2 y_5 + y_0 y_2 y_4 + y_0 y_2 y_3 + y_0 y_1 y_5 + y_0 y_1 y_4 + y_0 y_1 y_2 + y_5 y_6 + y_4 y_6 + y_3 y_4 + y_2 y_6 + y_1 y_2 \\
& + y_0 y_6 + y_0 y_2 + y_0 y_1 + y_6 + y_2 + y_1 + y_0, \\
x_2 = & y_1 y_2 y_3 y_5 y_6 + y_0 y_1 y_2 y_3 y_5 + y_1 y_3 y_5 y_6 + y_1 y_2 y_4 y_6 + y_1 y_2 y_3 y_6 + y_1 y_2 y_3 y_4 + y_0 y_2 y_4 y_6 + y_0 y_2 y_4 y_5 \\
& + y_0 y_2 y_3 y_6 + y_0 y_1 y_4 y_5 + y_4 y_5 y_6 + y_3 y_5 y_6 + y_2 y_4 y_6 + y_2 y_3 y_6 + y_2 y_3 y_5 + y_1 y_5 y_6 + y_1 y_4 y_5 \\
& + y_1 y_3 y_5 + y_1 y_3 y_4 + y_1 y_2 y_6 + y_0 y_4 y_5 + y_0 y_2 y_6 + y_0 y_2 y_5 + y_0 y_2 y_4 + y_0 y_2 y_3 + y_0 y_1 y_5 + y_0 y_1 y_4 \\
& + y_0 y_1 y_3 + y_4 y_6 + y_3 y_6 + y_3 y_5 + y_3 y_4 + y_2 y_6 + y_1 y_3 + y_1 y_2 + y_0 y_5 + y_0 y_4 + y_0 y_3 + y_0 y_2 \\
& + y_0 y_1 + y_6 + y_5 + y_4 + y_2 + y_1, \\
x_3 = & y_1 y_2 y_4 y_5 y_6 + y_2 y_4 y_5 y_6 + y_2 y_3 y_5 y_6 + y_1 y_3 y_4 y_6 + y_1 y_2 y_5 y_6 + y_0 y_4 y_5 y_6 + y_0 y_2 y_4 y_5 + y_0 y_2 y_3 y_6 \\
& + y_0 y_1 y_3 y_4 + y_3 y_5 y_6 + y_2 y_5 y_6 + y_2 y_4 y_5 + y_1 y_5 y_6 + y_1 y_4 y_6 + y_1 y_3 y_4 + y_1 y_2 y_6 + y_0 y_5 y_6 \\
& + y_0 y_4 y_5 + y_0 y_3 y_6 + y_0 y_2 y_5 + y_0 y_2 y_4 + y_0 y_2 y_3 + y_0 y_1 y_6 + y_0 y_1 y_4 + y_0 y_1 y_2 + y_5 y_6 + y_2 y_5 \\
& + y_2 y_4 + y_2 y_3 + y_1 y_4 + y_1 y_2 + y_0 y_6 + y_0 y_5 + y_0 y_4 + y_0 y_3 + y_0 y_2 + y_0 y_1 + y_6 + y_2 + y_1 + y_0, \\
x_4 = & y_0 y_1 y_3 y_5 y_6 + y_0 y_1 y_2 y_4 y_5 + y_1 y_3 y_4 y_6 + y_1 y_2 y_5 y_6 + y_1 y_2 y_4 y_5 + y_0 y_3 y_4 y_6 + y_0 y_2 y_4 y_5 + y_0 y_1 y_5 y_6 \\
& + y_0 y_1 y_3 y_5 + y_0 y_1 y_3 y_4 + y_0 y_1 y_2 y_4 + y_0 y_1 y_2 y_3 + y_3 y_4 y_6 + y_3 y_4 y_5 + y_2 y_4 y_5 + y_1 y_4 y_6 + y_1 y_2 y_6 \\
& + y_1 y_2 y_5 + y_1 y_2 y_3 + y_0 y_4 y_6 + y_0 y_4 y_5 + y_0 y_2 y_6 + y_0 y_2 y_5 + y_0 y_2 y_4 + y_0 y_2 y_3 + y_0 y_1 y_5 + y_0 y_1 y_4 \\
& + y_4 y_6 + y_3 y_4 + y_2 y_6 + y_2 y_4 + y_2 y_3 + y_0 y_5 + y_0 y_4 + y_0 y_3 + y_0 y_2 + y_0 y_1 + y_6 + y_5 + y_3 + y_1 + y_0, \\
x_5 = & y_0 y_2 y_3 y_4 y_5 + y_0 y_1 y_3 y_4 y_5 + y_3 y_4 y_5 y_6 + y_2 y_3 y_4 y_5 + y_1 y_3 y_4 y_6 + y_1 y_3 y_4 y_5 + y_1 y_2 y_5 y_6 + y_1 y_2 y_4 y_5 \\
& + y_1 y_2 y_4 y_5 + y_0 y_3 y_4 y_5 + y_0 y_2 y_3 y_6 + y_0 y_1 y_3 y_5 + y_3 y_5 y_6 + y_3 y_4 y_6 + y_2 y_5 y_6 + y_2 y_4 y_6 + y_2 y_4 y_5 \\
& + y_2 y_3 y_5 + y_1 y_4 y_6 + y_1 y_3 y_5 + y_0 y_5 y_6 + y_0 y_4 y_6 + y_0 y_4 y_5 + y_0 y_3 y_6 + y_0 y_1 y_6 + y_0 y_1 y_5 + y_5 y_6 \\
& + y_4 y_6 + y_3 y_4 + y_4 + y_2, \\
x_6 = & y_0 y_2 y_4 y_5 y_6 + y_0 y_1 y_3 y_4 y_6 + y_2 y_4 y_5 y_6 + y_2 y_3 y_5 y_6 + y_0 y_4 y_5 y_6 + y_0 y_2 y_5 y_6 + y_0 y_2 y_4 y_6 + y_0 y_2 y_4 y_5 \\
& + y_0 y_2 y_3 y_6 + y_0 y_2 y_3 y_5 + y_0 y_1 y_4 y_6 + y_0 y_1 y_4 y_5 + y_0 y_1 y_3 y_4 + y_0 y_1 y_2 y_6 + y_4 y_5 y_6 + y_3 y_5 y_6 \\
& + y_3 y_4 y_6 + y_2 y_5 y_6 + y_2 y_4 y_6 + y_2 y_4 y_5 + y_2 y_3 y_6 + y_2 y_3 y_4 + y_1 y_5 y_6 + y_1 y_4 y_5 + y_1 y_3 y_4 + y_0 y_5 y_6 \\
& + y_0 y_4 y_6 + y_0 y_4 y_5 + y_0 y_3 y_6 + y_0 y_3 y_5 + y_0 y_2 y_6 + y_0 y_2 y_5 + y_0 y_2 y_4 + y_0 y_2 y_3 + y_0 y_1 y_6 + y_0 y_1 y_3 \\
& + y_0 y_1 y_2 + y_5 y_6 + y_4 y_5 + y_3 y_6 + y_3 y_4 + y_2 y_5 + y_2 y_4 + y_2 y_3 + y_1 y_6 + y_1 y_5 + y_1 y_4 + y_1 y_3 \\
& + y_1 y_2 + y_0 y_5 + y_0 y_4 + y_0 y_3 + y_0 y_2 + y_0 y_1 + y_6 + y_4 + y_3 + y_2 + y_1 + y_0.
\end{aligned}$$

The algebraic degree of the inverse of the S-box is 5, with each linear combination of x_i reaching this maximum degree.

4.3.3 S-box Linearization

Affine subspaces of size up to 4, as to be shown below, could be found so that the S-box can be linearized. Note that the S-box is the only nonlinear mapping of the round function. Hence, the entire round function becomes linear when all S-boxes are restricted to such subspaces.

Definition 6 (Linearizable affine subspace [GLL⁺19]). *Linearizable affine subspaces are affine input subspaces on which S-box substitution is equivalent to a linear transformation. If V is a linearizable affine subspace of an S-box operation $\mathcal{S}(\cdot)$, then $\forall x \in V, \mathcal{S}(x) = A \cdot x + b$, where A is a 7×7 matrix and b is a 7-bit constant vector.*

Search for the linearizable affine subspaces of the S-box shows:

Observation 1. *Out of the entire 7-dimensional input space,*

- a. There are totally 749 2-dimensional linearizable affine subspaces.*
- b. There does not exist any linearizable affine subspace with dimension 3 or more.*

For completeness, any 1-dimensional subspace is always linearizable.

Noting that the S-box does not have linearizable affine subspace with dimension 3 or more, which is the same as SHA-3's S-box χ . The TDA [DDS12] and TIDA [ZHL24] algorithms used in SHA-3's security analysis can not gain additional advantage in analysing QILIN.

Linearizable affine subspaces with fixed input and output differences are of particular significance for the TDA and TIDA algorithms. We observe the relationship between linearizable affine subspaces and the DDT of the S-box:

Observation 2. *Given a 7-bit input difference δ_{in} and a 7-bit output difference δ_{out} such that $\text{DDT}(\delta_{in}, \delta_{out}) \neq 0$, i.e., the solution set $V = \{x : \mathcal{S}(x) + \mathcal{S}(x + \delta_{in}) = \delta_{out}\}$ is not empty, we have:*

- a. If $\text{DDT}(\delta_{in}, \delta_{out}) = 2$ or 4 , then V is a linearizable affine subspace.*
- b. If $\text{DDT}(\delta_{in}, \delta_{out}) = 6$, then there are three 2-dimensional subsets $V_i \subset V$, $i = 0, 1, 2$, such that V_i are linearizable affine subspaces.*
- c. If $\text{DDT}(\delta_{in}, \delta_{out}) = 8$, then there are six 2-dimensional subsets $V_i \subset V$, $i = 0, 1, \dots, 5$, such that V_i are linearizable affine subspaces.*

The 2-dimensional linearizable affine subspaces obtained from the analysis of DDT cover all the 749 cases in Observation 1.

4.3.4 Expected Size of Collision Subsets for the $\mathcal{S}$ S-box

The size of a collision subset is an important parameter in the birthday-search stage of internal-differential collision attacks. Following the probabilistic estimate used for Keccak in [ZHL24], we record here the corresponding local estimate for the 7-bit $\mathcal{S}$ S-box of QILIN. This calculation should be read as a $\mathcal{S}$ -local estimate: a full collision attack must additionally specify the internal-difference family before the last Q layer and the mapping from selected output lanes to the corresponding S-output components.

Recall that $\mathcal{S}$ is a permutation over $\mathbb{F}_2^7$. Its maximum differential probability is $8/128 = 2^{-4}$, and the maximum absolute entry in its LAT is 40. Moreover, for all nonzero input differences, the positive entries in the DDT only take the values 2, 4, 6, 8. Therefore, for partial outputs of $\mathcal{S}$, the collision probability depends on the selected output components and on the input difference; a geometric averaging method gives a more accurate estimate than treating all partial outputs as uniformly distributed.

For $1 \leq j \leq 7$, let $C_j = \{0, 1, \dots, j-1\}$ be the first j output components of $\mathcal{S}$, and let $\mathcal{S}_{C_j}$ denote the projection of $\mathcal{S}$ to these components. For an input difference $\Delta \in \mathbb{F}_2^7$, define

$$P_{\Delta}^{(j)} = \Pr_{a, a' \leftarrow \mathbb{F}_2^7} [(\mathcal{S}_{C_j}(a), \mathcal{S}_{C_j}(a \oplus \Delta)) = (\mathcal{S}_{C_j}(a'), \mathcal{S}_{C_j}(a' \oplus \Delta))].$$

The zero difference is included in the following average, as in the Table-4-style estimate of [ZHL24]. We define

$$P_{\text{geo}}^{(j)} = \left(\prod_{\Delta \in \mathbb{F}_2^7} P_{\Delta}^{(j)} \right)^{1/128}, \quad P_{\text{arith}}^{(j)} = \frac{1}{128} \sum_{\Delta \in \mathbb{F}_2^7} P_{\Delta}^{(j)}.$$

Using the same period-32 paired-state convention as the collision-subset estimate in [ZHL24], the expected sizes are

$$N_{\text{geo}}^{(j)} = (P_{\text{geo}}^{(j)})^{-32}, \quad N_{\text{arith}}^{(j)} = (P_{\text{arith}}^{(j)})^{-32}.$$

If all 64 bit positions of a lane are modeled as independent positions, the exponents in Table 4.6 should be doubled.

Table 4.6: The $\mathcal{S}$ -local expected size for collision length no more than j lanes

Collision length	$\mathcal{S}$ -output components C_j	G.E. size	A.E. size
1 lane	$\{0\}$	$2^{62.82}$	$2^{62.62}$
2 lanes	$\{0, 1\}$	$2^{121.19}$	$2^{120.22}$
3 lanes	$\{0, 1, 2\}$	$2^{169.57}$	$2^{167.63}$
4 lanes	$\{0, 1, 2, 3\}$	$2^{202.69}$	$2^{201.06}$
5 lanes	$\{0, 1, 2, 3, 4\}$	$2^{217.75}$	$2^{217.16}$
6 lanes	$\{0, 1, 2, 3, 4, 5\}$	$2^{222.76}$	$2^{222.65}$
7 lanes	$\{0, 1, 2, 3, 4, 5, 6\}$	$2^{224.00}$	$2^{224.00}$

In the last row, C_7 contains all output components of $\mathcal{S}$. Since $\mathcal{S}$ is a permutation, for every Δ the pair $(\mathcal{S}(a), \mathcal{S}(a \oplus \Delta))$ has exactly 128 possible values as a ranges over $\mathbb{F}_2^7$, and hence $P_\Delta^{(7)} = 2^{-7}$. This gives $(2^{-7})^{-32} = 2^{224}$, which is consistent with the full-output case in Table 4.6

Digest-prefix lane mapping

We now apply the same local estimate to the actual output prefixes of QILIN. The state is serialized as

$$S = x_{0,0} \| x_{1,0} \| \cdots \| x_{6,0} \| x_{0,1} \| \cdots \| x_{6,6}.$$

Hence the n -th output lane has coordinate

$$(x, y) = (n \bmod 7, \lfloor n/7 \rfloor).$$

The round-function specification gives, for every $i \in \{0, \dots, 6\}$,

$$(x_{i,0}, x_{i+1,1}, \dots, x_{i+6,6}) \leftarrow Q(y_{i,-i}, y_{i,1-i}, \dots, y_{i,6-i}),$$

where all indices are modulo 7. Therefore the component-wise lane correspondence of Q_i is

$$\text{input component } c : y_{i,c-i}, \quad \text{output component } c : x_{i+c,c}.$$

Equivalently, an output lane (x, y) is produced by

$$Q_{(x-y) \bmod 7} \quad \text{and corresponds to output component } y.$$

For a selected output-lane set O , define the component set observed from Q_i by

$$C_i(O) = \{y : ((i + y) \bmod 7, y) \in O\}.$$

Table 4.7: The Q_i -component coverage of the digest prefixes

Output length	Selected lanes	Q_i -component coverage
512 bits	8	$Q_0, \dots, Q_5 : \{0\}, Q_6 : \{0, 1\}$
768 bits	12	$Q_0, Q_1, Q_2, Q_3, Q_6 : \{0, 1\}, Q_4, Q_5 : \{0\}$
1024 bits	16	$Q_0, \dots, Q_4 : \{0, 1\}, Q_5, Q_6 : \{0, 1, 2\}$

Table 4.7 shows that all three digest prefixes touch all seven Q_i boxes. Consequently, at the level immediately before the last nonlinear layer, the union of the input lanes involved in these prefixes is the full 7×7 lane state. The distinction between the 512-, 768-, and 1024-bit prefixes is instead how many output components of each Q_i are observed.

Let

$$e_{\text{geo}}(C) = -\log_2 P_{\text{geo}}^C, \quad e_{\text{arith}}(C) = -\log_2 P_{\text{arith}}^C.$$

For a digest prefix O , the per-bit-slice exponent is the sum of the corresponding local exponents over the seven Q_i boxes:

$$E_{\text{geo}}(O) = \sum_{i=0}^6 e_{\text{geo}}(C_i(O)), \quad E_{\text{arith}}(O) = \sum_{i=0}^6 e_{\text{arith}}(C_i(O)).$$

Under the same period-32 paired-state convention used above, the expected collision-subset sizes are

$$N_{\text{geo}}(O) = 2^{32E_{\text{geo}}(O)}, \quad N_{\text{arith}}(O) = 2^{32E_{\text{arith}}(O)}.$$

Here A.E. follows the convention of [ZHL24]: the local collision probabilities are averaged arithmetically first, and the resulting averaged probability is then inverted.

Table 4.8: Average expected collision-subset sizes for digest prefixes

Output length	E_{geo}	G.E. size	E_{arith}	A.E. size
512 bits	15.5662	$2^{498.12}$	15.4987	$2^{495.96}$
768 bits	22.8620	$2^{731.58}$	22.6981	$2^{726.34}$
1024 bits	29.5335	$2^{945.07}$	29.2610	$2^{936.35}$

The corresponding per-slice geometric exponents are

$$\begin{aligned} E_{\text{geo}}(512) &= 6e_{\text{geo}}(\{0\}) + e_{\text{geo}}(\{0, 1\}), \\ E_{\text{geo}}(768) &= 2e_{\text{geo}}(\{0\}) + 5e_{\text{geo}}(\{0, 1\}), \\ E_{\text{geo}}(1024) &= 5e_{\text{geo}}(\{0, 1\}) + 2e_{\text{geo}}(\{0, 1, 2\}). \end{aligned}$$

The arithmetic exponents are obtained by replacing e_{geo} with e_{arith} in the same formulas. If all 64 bit positions of each lane are modeled as independent positions, the exponents in Table 4.8 should be doubled.

4.3.5 Integral Distinguishers and Division Property

Utilizing Todo’s division property [Tod15], integral distinguishers for the 3136-bit QILIN- f permutation are found to cover up to 5 rounds, with complexity bounded by 2^{1024} , as shown in Table 4.9. Similarly, 5-round integral distinguishers are found for the inverse of QILIN- f with complexity bounded by 2^{1024} , as shown in Table 4.10.

Table 4.9: Integral distinguishers for QILIN- f

#Rounds	2	3	4	5	6
$\log_2(\text{data})$	7	21	68	259	1029

Table 4.10: Integral distinguishers for the inverse of QILIN- f

#Rounds	2	3	4	5	6
$\log_2(\text{data})$	7	28	126	630	2689

4.3.6 Zero-Sum Distinguisher

A zero-sum distinguisher [BC10, BCC11] on the 3136-bit QILIN- f permutation with $r = r_f + r_b$ rounds is built around an intermediate state, by selecting an affine subspace V of dimension $d = \dim V$, propagating it forward by r_f rounds and backward by r_b rounds, and exploiting that

$$\bigoplus_{x \in V} R^{-r_b}(x) = 0 \text{ and } \bigoplus_{x \in V} R^{r_f}(x) = 0, \text{ if } d > \max(D^+(r_f), D^-(r_b)),$$

where R is the Round function and $D^+(r_f)$ (resp. $D^-(r_b)$) is an upper bound on the algebraic degree of r_f forward (resp. r_b inverse) rounds of QILIN- f .

Each output bit of Q has algebraic degree of 4, which gives an upper bound on $D^+(r_f) \leq 4^{r_f}$. In particular, we have $D^+(4) \leq 256$. For the inverse direction, each output bit of Q^{-1} has algebraic degree of 5. Hence one inverse round multiplies the algebraic degree by at most 5, giving the upper bounds $D^-(r_b) \leq 5^{r_b}$, in particular, $D^-(4) \leq 625$. Combining these two parts, a zero-sum distinguisher with complexity bounded by 2^{1024} can achieve the number of rounds:

$$r = r_f + r_b = 4 + 4 = 8.$$

If we combine it with the other techniques, such as division property [Tod15] and linearization techniques [GLS16], one more round can be achieved. That is, zero-sum distinguishers can be obtained for 9 rounds with complexities bounded by 2^{1024} .

4.3.7 Statistical Randomness Testing

The statistical randomness of the proposed hash function QILIN and the permutation QILIN- f was evaluated using both the NIST SP 800-22 and the GM/T 0005-2012 test suites. We utilized counter-based inputs, random inputs, and low Hamming weight inputs. Experimental results demonstrate that both primitives successfully pass all tests in these suites. Notably, it takes only three rounds for QILIN- f to satisfy all randomness requirements.

4.4 Analysis of the Algorithm

4.4.1 Collision Attack

Internal differential attacks

We evaluate the resistance of QILIN against reduced-round collision attacks using the internal-differential methodology [DDS13, ZHL23, ZHL24], which was developed for security analysis of SHA-3. The generalized internal differentials of Dinur, Dunkelman and Shamir map a large set of inputs into a smaller subset of possible outputs and then exploit a birthday search inside this reduced target subset [DDS13]. The later conditional internal-differential framework refines the connection to the message space by imposing linear conditions on the initial messages and by partitioning last-round candidates according to the values of these conditions [ZHL23]. The probabilistic-linearization approach further relaxes deterministic S-box linearization, uses high-density affine subspaces for local differences, and gives a more accurate estimate of collision-subset sizes at the digest level [ZHL24]. Following this line, we adapt TIDA-style internal-differential search to the QILIN sponge construction and to the $7 \times 7 \times 64$ -bit permutation. In our model, the capacity part is fixed according to the target QILIN instance, while the bitrate/message part provides the degrees of freedom used to satisfy the internal transition constraints. We search for period-32 internal differential paths, connect them either by TIDA or by direct constrained enumeration, and finally apply a birthday argument on the effective output subset. This yields collision attacks on 3-round QILIN-512 and 2-round QILIN-768, but not for 2-round QILIN-1024. We are unable to extend the collision attacks to more rounds by using the internal differentials.

3-round collision attack on QILIN-512

We describe a theoretical collision attack on three rounds of QILIN-512. The attack uses the period-32 internal differential path with

$$\text{start_round} = 0, \quad AS_0 = 177.$$

The path is

$$z_0 \longrightarrow y_1 \longrightarrow z_1 \longrightarrow y_2,$$

where the corresponding round-constant differences are

$$RC_0 = 0x01027fd7, \quad RC_1 = 0x01027fd6, \quad RC_2 = 0x01027fd5.$$

For this path we have

$$AS_0 = 177, \quad AS_1 = 14, \quad AS_2 = 0.$$

The active bit positions in z_1 are

$$\{1, 2, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 17, 24\},$$

with the following nonzero input differences for the active local Q -boxes:

bit	1	2	4	6	7	8	9	10	11	12	13	14	17	24
Δ_{in}	06	12	16	16	16	16	06	12	10	12	16	16	12	06

The first step is to use TIDA to generate a valid value of y_0 . For this path the optimized affine-subspace construction gives

$$w \simeq 0.99, \quad \#E_1 = 1037, \quad \text{rank}(E_2) = 943,$$

and therefore $\text{Obj} = w + \#E_1 - \text{rank}(E_2) = 94.99$. Thus, one valid y_0 is obtained after about $2^{94.99} \simeq 2^{95}$ TIDA trials on average.

For each valid y_0 , the first-round constraints define a set of messages that satisfy the first-round internal differential. The exact expectation from the local distribution is $\mathbb{E}[|\mathcal{M}(y_0)|] = 2^{64.65}$. This value is consistent with the simulation over 2^{20} samples, where the observed average was $2^{63.72}$; in the following estimate we use the theoretical expectation $2^{64.65}$.

The second-round nonlinear transition $y_1 \rightarrow z_1$ has differential probability $\Pr[y_1 \rightarrow z_1] = 2^{-71}$. In the three-round setting, the final active parameter is $AS_2 = 0$. Hence the birthday part for a 512-bit output is $2^{(0+256)/2} = 2^{128}$. Following the same accounting convention as in the path filtering stage, the number of messages satisfying the first-round internal differential that must be collected is therefore $2^{71+(0+256)/2} = 2^{199}$.

Since each valid y_0 contributes, on average, a message space of size $2^{64.65}$, the required number of valid y_0 's is $2^{199-64.65} = 2^{134.35}$. Multiplying this by the average TIDA cost per valid y_0 , we obtain the total TIDA complexity $2^{134.35} \cdot 2^{94.99} \simeq 2^{229.34}$. Using the rounded quantities 2^{95} and $2^{64.65}$ gives the estimate $2^{199-64.65+95} = 2^{229.35}$. The subsequent processing of 2^{199} first-round-valid messages is smaller than the TIDA cost, so the overall theoretical complexity is dominated by $2^{229.34}$, which is below the generic birthday bound 2^{256} for QILIN-512 by about 26.66 bits.

This estimate assumes that independently generated valid y_0 's give essentially independent message spaces with negligible overlap, and that the expected value $2^{64.65}$ is the appropriate asymptotic average.

2-round collision attack on QILIN-768

We describe a theoretical collision attack on two rounds of QILIN-768. The attack uses the period-32 internal differential path

$$x_0 \longrightarrow y_0 \longrightarrow z_0 \longrightarrow x_1 \longrightarrow y_1$$

with

$$\text{start_round} = 1, \quad RC_1 = 0x01027fd6.$$

For this path,

$$AS_0 = 20, \quad AS_1 = 168, \quad 2AS_0 + AS_1 = 208.$$

In the period-32 internal model the state size is $7 \cdot 7 \cdot 32 = 1568$. For QILIN-768, the capacity constraint fixes 768 internal coordinates, so a single-block message provides $1568 - 768 = 800$ free internal coordinates. The concrete path was found in a stricter subspace, but its nonzero coordinates are contained in this 800-coordinate free part, so it is compatible with the QILIN-768 padding and capacity constraint.

For the first nonlinear layer, the local differential transition is $y_0 \longrightarrow z_0$. The DDT count histogram over the period-32 local boxes is

DDT count	2	4	8	128
number of boxes	2	6	12	204

and therefore

$$\begin{aligned}\Pr[y_0 \rightarrow z_0] &= \left(\frac{2}{128}\right)^2 \left(\frac{4}{128}\right)^6 \left(\frac{8}{128}\right)^{12} \\ &= 2^{-90}.\end{aligned}$$

By imposing these transition conditions directly on the initial message space, instead of using a TIDA stage, the expected number of messages that deterministically satisfy the first-round internal differential is $2^{800} \cdot 2^{-90} = 2^{710}$.

After the linear layer of the second round, the next nonlinear layer maps y_1 to a set of possible output differences. Restricted to the first 768 output bits, the number of possible truncated differences is 2^m , $m = 281.36$, which equals $2^{256} \cdot 3^{16}$ in the present computation. Hence, using the same truncated-difference birthday accounting as in the path-search filtering stage, the number of first-round-valid messages needed for a two-round collision is

$$\begin{aligned}N &= 2^{(384+m)/2} \\ &= 2^{(384+281.36)/2} \\ &= 2^{332.68}.\end{aligned}$$

The available first-round-valid message space is much larger, since $2^{710}/2^{332.68} = 2^{377.32}$. Thus the single-block QILIN-768 message space has sufficient freedom for this attack.

The attack complexity is dominated by generating and processing $2^{332.68}$ messages satisfying the first-round internal differential. Under the above assumptions, the total theoretical complexity is therefore $2^{332.68}$, which is below the generic birthday bound 2^{384} for QILIN-768 by about $384 - 332.68 = 51.32$ bits.

This estimate assumes that the constrained message generation can enumerate the first-round-valid solution set directly, that the local differential conditions behave with the DDT probability stated above, and that the truncated differences after the second nonlinear layer are sufficiently uniform for the birthday accounting to apply.

4.4.2 Preimage attack

Meet-in-the-middle attacks

We also evaluate the resistance of QILIN against preimage attacks, following the meet-in-the-middle (MitM) framework [QHD⁺23, DZQ⁺24]. Since the hash output takes only a single permutation call in QILIN (Case I of framework in Section 3 [DZQ⁺24]), we can only focus on the Phase II of the framework where two MitM attacks are performed and match at the capacity part. We re-implemented the framework in MILP language for QILIN, and the unit level is set as lane (also implemented in the bit level, but inefficient). As a result, for the lane model, we find a solution with advantage of one lane (64 bits) up to 2 rounds of QILIN. But, for 3 rounds, the lane model has no solution when we exhaustive all possibilities of lanes in the capacity part.

Rotational cryptanalysis

Rotation cryptanalysis [MPS13] has been successfully applied to round-reduced Keccak up to 4 rounds for preimage and 5 rounds for distinguisher. We adapt the framework to QILIN. For a rotation amount $\tau \in \{0, \dots, 63\}$, two states $(A, A^\leftarrow)$ form a *rotational pair* iff $A_{i,j,b}^\leftarrow = A_{i,j,(b-\tau) \bmod 64}$, and the per-bit disagreement probability $p_{i,j,b}^\tau := \Pr[\text{state}_{i,j,b} \neq \text{state}_{i,j,(b+\tau) \bmod 64}^\leftarrow]$ is tracked through each operation by the XOR, AND et al. propagation rules [MPS13].

We first directly derive Q 's rotational transfer in a single step by parameterizing the rotational-relation-breaking pattern of the seven input lanes as a vector $F \in \{0, 1\}^7$, with $F_k = 1$ iff lane k violates the rotational relation ($X' = X \oplus F$).

Proposition 1 (Rotational transfer of Q). *Let $\text{DDT}_i(F) := |\{X \in \{0, 1\}^7 : Q(X)_i \neq Q(X \oplus F)_i\}|$ for $i \in \{0, \dots, 6\}$, $F \in \{0, 1\}^7$. Under lane-independent input biases $\pi = (p_{a_0}, \dots, p_{a_6})$, the output biases $Q^*(\pi) = (p_{b_0}, \dots, p_{b_6})$ are*

$$p_{b_i} = \frac{1}{128} \sum_{F \in \{0, 1\}^7} \Pr(F) \cdot \text{DDT}_i(F), \quad \Pr(F) = \prod_{k=0}^6 p_{a_k}^{F_k} (1 - p_{a_k})^{1-F_k}.$$

We applied Proposition 1 of Q together with XOR rule (for L) to QILIN. As a result, we can find the rotational preimage attack on QILIN up to 2 rounds and rotational distinguisher on 3 rounds.

Preimage attacks on 2-round QILIN. There are 1218 positions remain deterministic ($p_{i,j,b}^\tau \in \{0, 1\}$) for every $\tau \in \{1, \dots, 63\}$; guessing $A^\leftarrow$ rather than A succeeds with probability $64 \cdot 2^{-n}$, giving

$$T = 2^{n-6}, \quad M = \mathcal{O}(1),$$

for every QILIN- n , $n \in \{512, 768, 1024\}$.

For 3-round QILIN, we can only find a distinguishing attack, whose maximum bias is $\max_{\tau,i,j,b} |p_{i,j,b}^\tau - 1/2| = 9/512$ at $\tau = 1$, and thus requiring $m \geq (p - 1/2)^{-2} \ln(1/\sqrt{\pi}) \approx 4.85 \times 10^3$ rotational pairs ($\pi = 0.05$).

4.4.3 Quantum Security

Indifferentiability

QILIN's construction is based on the sponge construction and achieves the same indifferentiability from a Random Oracle as the sponge construction for a classical adversary [BDPA08]. We believe that our construction will behave as secure as the sponge construction in the quantum-query context. The most recent result, which also provides the first proof of the quantum indifferentiability of the sponge construction instantiated with a random permutation, is due to [ACMT25]:

Proposition 2. *There exists an efficient simulator $\mathcal{S}$ such that for all adversaries $\mathcal{A}$ making q quantum queries each of length at most $r \cdot \ell$,*

$$\left| \Pr[\mathcal{A}^{\varphi, \varphi^{-1}, \text{Sp}^\varphi}() = 1] - \Pr[\mathcal{A}^{\mathcal{S}^f, f}() = 1] \right| = O\left(\ell^3 \sqrt[4]{q^9 2^{-\min(r,c)}}\right).$$

In the above, φ is a uniformly random permutation while f is a uniformly random function with the same domain and range as Sp^φ .

Here Sp^φ denotes the full sponge, i.e., both the input and the output are of arbitrary length (Based on our understanding, here's "arbitrary" means "multiple of r "). The corresponding indifferentiability is thus $O(2^{\min(r,c)/9})$.

Preimage and collision resistance

[ACMT25] also gives the quantum preimage and collision resistance of the sponge with a single round of squeezing (but arbitrary rounds of absorbing):

Proposition 3. *The probability that a quantum algorithm $\mathcal{A}$ making no more than q quantum queries to a random permutation $\varphi \in S_{\{0,1\}^{r+c}}$ and its inverse φ^{-1} outputs a sponge collision is upper bounded as*

$$\Pr_{\varphi \leftarrow S_{\{0,1\}^n}; m, m' \leftarrow \mathcal{A}} [m \neq m' \wedge \text{Sp}^\varphi(m) = \text{Sp}^\varphi(m')] \leq O\left(q^5 n 2^{-\min(r,c)}\right).$$

The same bound also holds for preimage-resistance. For constant success probability, $\tilde{\Omega}(2^{\min(r,c)/5})$ quantum queries are thus necessary to find a collision or a preimage.

Here the Sp^φ has an output length of r . According to our understanding of their method, the result will be $\tilde{\Omega}(2^{\min(\min(r,c)/5, d/3)})$ for the sponge with an output length of d ($d \leq r$).

Generic quantum preimage and collision attacks

QILIN has a very similar construction to the sponge construction and shares same security against classical generic preimage and collision attacks. For generic quantum preimage and collision attacks, a sponge collision can be found in $O(2^{\min(d,c)/3})$ quantum queries using the BHT algorithm for collision finding in case $d < c$, and the BHT algorithm for claw finding in case $c \leq d$ ("meet in the middle"). A preimage can be found using $O(\min(2^{c/3}, 2^{d/2}))$ queries using Grover or claw-finding BHT. QILIN has a capacity c of length $2d$, the corresponding quantum security-strength becomes $O(2^{d/3})$ and $O(2^{d/2})$ for collision and preimage attacks, i.e. $2^{170.7}$, 2^{256} , $2^{341.3}$ and 2^{256} , 2^{384} , 2^{512} for $d = 512, 768, 1024$.

Quantum collision attack on 3-round QILIN-512

We now estimate the quantum complexity of the 3-round collision attack on QILIN-512 described in Section 4.4.1. The estimate in this subsection is an optimistic coherent-oracle estimate. In particular, we assume that the relevant classical predicates and samplers used in the attack can be implemented reversibly and queried in superposition: the TIDA success predicate, the first-round message-space sampler attached to a valid y_0 , the second-round differential predicate, and the final effective collision tag. We also assume quantum-readable access to the tables required by the BHT collision-finding procedure. Thus the complexity below should be interpreted as a quantum time/query estimate, not as a low-memory sequential data-generation attack.

For the period-32 internal differential path used in Section 4.4.1, the relevant classical parameters are

$$p_{\text{TIDA}} = 2^{-94.99}, \quad \mu_1 = 2^{64.65}, \quad p_2 = 2^{-71}, \quad AS_2 = 0.$$

Here p_{TIDA} is the success probability of one TIDA trial for producing a valid value of y_0 , μ_1 is the expected number of first-round-valid messages contributed by one valid y_0 , and p_2 is the probability of the second-round nonlinear differential transition $y_1 \rightarrow z_1$.

The TIDA stage can be quantized by applying amplitude amplification to the classical TIDA trial. The underlying oracle only checks whether the affine systems constructed in TIDA are consistent and, if so, recovers a valid solution; this is a reversible computation over $\mathbb{F}_2$, not an application of a real/complex quantum linear-system algorithm. Therefore the cost of obtaining one valid y_0 is reduced from $p_{\text{TIDA}}^{-1} = 2^{94.99}$ classical trials to $C_{\text{TIDA}}^{(1)} = p_{\text{TIDA}}^{-1/2} = 2^{47.495}$ coherent TIDA trials, up to polynomial overhead for reversible linear algebra.

For the final collision step, the classical attack uses a birthday factor over an effective space of size

$$R_{\text{eff}} = 2^{AS_2+256} = 2^{256}.$$

In the quantum setting, we replace this birthday factor by the BHT collision-finding factor

$$B_Q = R_{\text{eff}}^{1/3} = 2^{(AS_2+256)/3} = 2^{256/3} \simeq 2^{85.33}.$$

Accordingly, in the same path-accounting convention as in the classical analysis, the number of first-round-valid messages represented by the coherent candidate domain is

$$N_1^Q = p_2^{-1} B_Q = 2^{71+85.33} = 2^{156.33}.$$

Since one valid y_0 contributes on average $\mu_1 = 2^{64.65}$ first-round-valid messages, the required number of valid y_0 's is

$$L_Q = \frac{N_1^Q}{\mu_1} = 2^{156.33-64.65} = 2^{91.68}.$$

Generating these L_Q valid y_0 's with amplitude amplification gives

$$\begin{aligned} C_{\text{TIDA}}^Q &= L_Q \cdot C_{\text{TIDA}}^{(1)} \\ &= 2^{91.68} \cdot 2^{47.495} \\ &= 2^{139.175} \simeq 2^{139.18}. \end{aligned}$$

The second-round differential filtering can also be implemented by amplitude amplification. Since the transition probability is $p_2 = 2^{-71}$, the cost of obtaining one second-round-valid candidate from the first-round-valid message space is reduced from $p_2^{-1} = 2^{71}$ to $p_2^{-1/2} = 2^{35.5}$. The BHT procedure makes $B_Q \simeq 2^{85.33}$ coherent candidate queries, so the corresponding second-round filtering cost is

$$C_2^Q = B_Q \cdot p_2^{-1/2} = 2^{85.33+35.5} = 2^{120.83}.$$

The BHT collision-finding step itself contributes

$$C_{\text{BHT}}^Q = B_Q = 2^{85.33}$$

queries to the final effective tag oracle, plus lower-order reversible comparison and table-lookup costs.

Consequently, under the above coherent-oracle assumptions, the total quantum time complexity is dominated by the TIDA preparation stage:

$$\begin{aligned} C_{\text{total}}^Q &= \max \left\{ C_{\text{TIDA}}^Q, C_2^Q, C_{\text{BHT}}^Q \right\} \\ &= \max \left\{ 2^{139.18}, 2^{120.83}, 2^{85.33} \right\} \\ &= 2^{139.18}. \end{aligned}$$

This should be compared with the generic quantum collision complexity $2^{512/3} \simeq 2^{170.67}$ for a 512-bit output. Hence, in the optimistic coherent-oracle model, the 3-round structural collision attack has a quantum time advantage of about $170.67 - 139.18 = 31.49$ bits over generic BHT collision search.

We emphasize the storage assumptions behind this estimate. First, the BHT algorithm requires a quantum-readable table of size

$$\Theta(B_Q) = \Theta(2^{85.33})$$

records, each storing an effective collision tag and the corresponding candidate descriptor. Second, the TIDA part of the estimate assumes coherent access to the $L_Q = 2^{91.68}$ valid y_0 -families, either through a quantum-readable precomputed table or through an ideal reversible generator with the same asymptotic access cost. If these y_0 -families are stored explicitly, the dominant storage term is therefore

$$\Theta(2^{91.68})$$

state-family records, in addition to the BHT table. Without such coherent access, or if all candidates must be generated and stored sequentially before the collision search, the above $2^{139.18}$ estimate no longer describes the actual end-to-end cost.

Chapter 5

Implementation

5.1 Software Implementation

The shifted XOR is fast on ARM CPUs. The ARM’s barrel shifter allows instructions to apply a range of constant shifts to the second operand register. We can use a single instruction to perform a shifted XOR operation. On other CPU architectures (such as x86_64), we have minimized the number of rotate operations as much as possible. We reduce the number of NOT operations by complementing transform (Section 3.2.2).

5.1.1 X86_64 Architecture

- CPU: Hygon Dharma Processor
- CPU frequency: 2700 MHz
- Memory size: 3.8 Gi
- OS: Ubuntu 24.04 LTS
- Kernel: 6.8.0-117-generic
- Compiler: gcc (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
- cmake version 3.28.3

Table 5.1: Times and cycles per byte of QILIN on X86_64 architecture

Message size	QILIN-512		QILIN-768		QILIN-1024	
	time (μs)	cpb	time (μs)	cpb	time (μs)	cpb
32B	1.295	103.1	1.459	117.1	1.616	130.2
128B	1.232	24.47	1.399	28.00	1.581	31.86
512B	2.300	11.76	3.954	20.47	5.760	30.00
1024B	4.354	11.30	7.491	19.56	11.19	29.33
4096B	16.76	11.00	25.31	16.64	42.61	28.04
8192B	33.41	10.99	49.17	16.18	83.21	27.40
16384B	65.13	10.72	98.44	16.21	164.7	27.13
65536B	257.1	10.59	398.7	16.42	654.2	26.95

5.1.2 ARM Architecture

- CPU: Neoverse-N1
- Memory size: 30 Gi

Table 5.2: Memory of QILIN on X86_64 architecture

	static memory (bytes)	peak memory (bytes)
QILIN-512	20480	20480
QILIN-768	20480	20480
QILIN-512	20480	20480

- OS: Ubuntu 24.04 LTS
- Kernel: 6.17.0-1017-aws
- Compiler: gcc (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
- cmake version 3.28.3

Table 5.3: Times and cycles per byte of QILIN on ARM architecture

Message size	QILIN-512		QILIN-768		QILIN-1024	
	time (μs)	cpb	time (μs)	cpb	time (μs)	cpb
32B	21.86	157.8	22.33	185.0	23.76	211.4
128B	21.55	37.69	22.27	45.96	22.16	51.71
512B	22.79	14.83	24.74	23.46	26.58	33.51
1024B	24.66	12.42	28.63	20.70	32.69	30.45
4096B	38.13	10.91	46.82	16.37	64.87	27.12
8192B	55.76	10.62	71.90	15.65	106.7	26.30
16384B	89.22	10.30	122.5	15.47	191.1	25.88
65536B	288.1	10.09	424.9	15.33	696.5	25.66

Table 5.4: Memory of QILIN on ARM architecture

	static memory (bytes)	peak memory (bytes)
QILIN-512	135168	200704
QILIN-768	135168	200704
QILIN-512	135168	200704

5.2 Hardware Implementation

We implemented QILIN in Verilog. The hardware architecture was synthesized and laid out using the OpenROAD-flow-scripts (ORFS) toolchain under the Nangate 45nm open-cell library. At a target frequency of 50 MHz (clock period of 20.0 ns), post-layout timing analysis shows that setup and hold constraints are fully satisfied with zero violations. The worst-case register-to-register setup slack is 18.42 ns, corresponding to a core critical path delay of only 1.9 ns (with a pure logic gate delay of 1.53 ns).

Table 5.5: Hardware performance of core of QILIN-512 on NanGate 45nm chip.

Algorithm	Gates	Area	Critical Path	Frequency	Throughput
QILIN-512	77796	$1.5 \times 10^5 \mu m^2$	1.9 ns	526 MHz	46.29 Gbit/s

Bibliography

- [ACMT25] Gorjan Alagic, Joseph Carolan, Christian Majenz, and Saliha Tokat. The sponge is quantum indifferentiable. In *66th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2025, Sydney, Australia, December 14-17, 2025*, pages 2591–2630. IEEE, 2025.
- [BC10] Christina Boura and Anne Canteaut. Zero-sum distinguishers for iterated permutations and application to Keccak-f and Hamsi-256. In *Selected Areas in Cryptography - SAC 2010*, volume 6544 of *Lecture Notes in Computer Science*, pages 1–17. Springer, 2010.
- [BCC11] Christina Boura, Anne Canteaut, and Christophe De Cannière. Higher-order differential properties of Keccak and Luffa. In *Fast Software Encryption - FSE 2011*, volume 6733 of *Lecture Notes in Computer Science*, pages 252–269. Springer, 2011.
- [BDP⁺12] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Keccak implementation overview. <http://keccak.noekeon.org/>, May 2012. Available at <http://keccak.noekeon.org/>.
- [BDPA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the sponge construction. In Nigel P. Smart, editor, *Advances in Cryptology - EUROCRYPT 2008, 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Istanbul, Turkey, April 13-17, 2008. Proceedings*, Lecture Notes in Computer Science, pages 181–197. Springer, 2008.
- [BR22] Tim Beyne and Vincent Rijmen. Differential cryptanalysis in the fixed-key model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part III*, Lecture Notes in Computer Science, pages 687–716. Springer, 2022.
- [CF25] Anne Canteaut and Merlin Fruchon. Understanding unexpected fixed-key differential behaviours: How to avoid major weaknesses in lightweight designs. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8-12, 2025, Proceedings, Part I*, Lecture Notes in Computer Science, pages 97–130. Springer, 2025.
- [DDS12] Itai Dinur, Orr Dunkelman, and Adi Shamir. New attacks on keccak-224 and keccak-256. In Anne Canteaut, editor, *Fast Software Encryption - 19th International Workshop, FSE 2012, Washington, DC, USA, March 19-21, 2012. Revised Selected Papers*, Lecture Notes in Computer Science, pages 442–461. Springer, 2012.
- [DDS13] Itai Dinur, Orr Dunkelman, and Adi Shamir. Collision attacks on up to 5 rounds of SHA-3 using generalized internal differentials. In Shiho Moriai, editor, *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers*, volume 8424 of *Lecture Notes in Computer Science*, pages 219–240. Springer, 2013.
- [DR07] Joan Daemen and Vincent Rijmen. Plateau characteristics. *IET Inf. Secur.*, 1(1):11–17, 2007.

- [DZQ⁺24] Xiaoyang Dong, Boxin Zhao, Lingyue Qin, Qingliang Hou, Shun Zhang, and Xiaoyun Wang. Generic mitm attack frameworks on sponge constructions. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV*, Lecture Notes in Computer Science, pages 3–37. Springer, 2024.
- [GLL⁺19] Jian Guo, Guohong Liao, Guozhen Liu, Meicheng Liu, Kexin Qiao, and Ling Song. Practical collision attacks against round-reduced SHA-3. *IACR Cryptol. ePrint Arch.*, 2019:147, 2019.
- [GLS16] Jian Guo, Meicheng Liu, and Ling Song. Linear structures: Applications to cryptanalysis of round-reduced keccak. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 249–274, 2016.
- [MPS13] Pawel Morawiecki, Josef Pieprzyk, and Marian Srebrny. Rotational cryptanalysis of round-reduced keccak. In Shihō Moriai, editor, *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers*, volume 8424 of *Lecture Notes in Computer Science*, pages 241–262. Springer, 2013.
- [NO14] Yusuke Naito and Kazuo Ohta. Improved indifferentiable security analysis of PHOTON. In Michel Abdalla and Roberto De Prisco, editors, *Security and Cryptography for Networks - 9th International Conference, SCN 2014, Amalfi, Italy, September 3-5, 2014. Proceedings*, Lecture Notes in Computer Science, pages 340–357. Springer, 2014.
- [QHD⁺23] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-middle preimage attacks on sponge-based hashing. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part IV*, volume 14007 of *Lecture Notes in Computer Science*, pages 158–188. Springer, 2023.
- [Tod15] Yosuke Todo. Structural evaluation by generalized integral property. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, Lecture Notes in Computer Science, pages 287–314. Springer, 2015.
- [YL25] Xiaobin Yu and Meicheng Liu. Multiple rows mixers and hsilu A family of linear layers and a permutation with fewer xors. *IACR Trans. Symmetric Cryptol.*, 2025(2):166–191, 2025.
- [ZHL23] Zhongyi Zhang, Chengan Hou, and Meicheng Liu. Collision attacks on round-reduced SHA-3 using conditional internal differentials. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part IV*, volume 14007 of *Lecture Notes in Computer Science*, pages 220–251. Springer, 2023.
- [ZHL24] Zhongyi Zhang, Chengan Hou, and Meicheng Liu. Probabilistic linearization: Internal differential collisions in up to 6 rounds of SHA-3. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV*, Lecture Notes in Computer Science, pages 241–272. Springer, 2024.